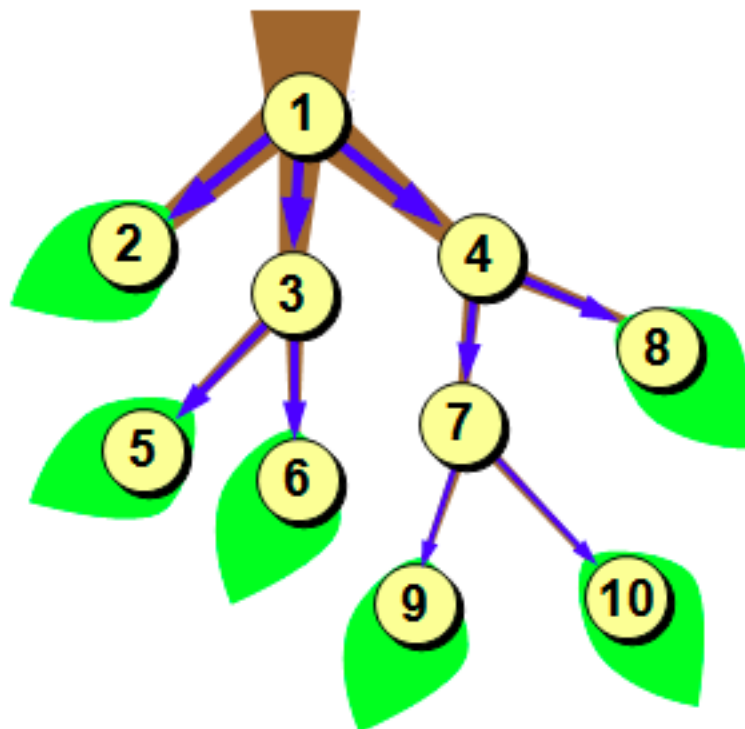




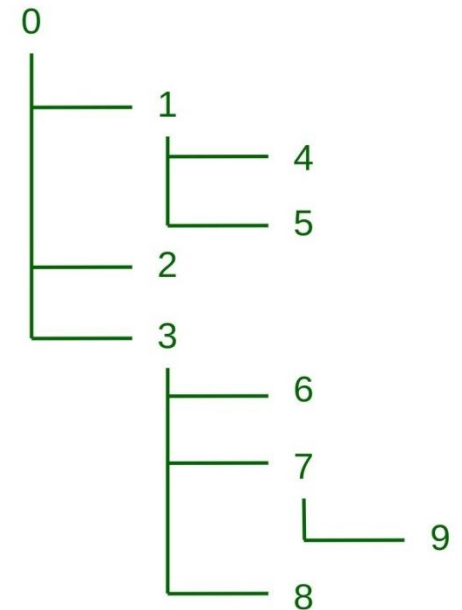
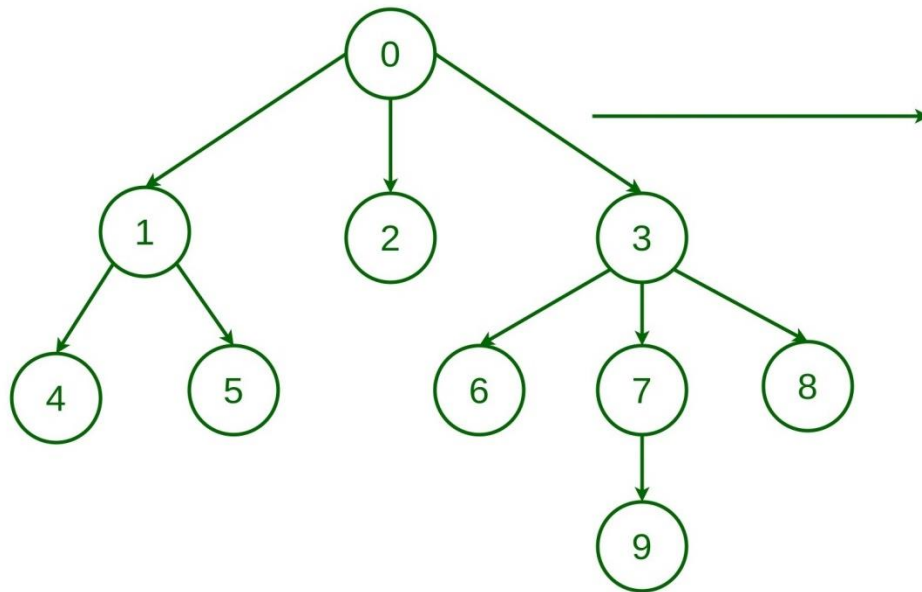
ТЕОРИЯ ГРАФОВ

Деревья

Деревья

- Деревья являются в некотором смысле простейшим классом графов. Для них *выполняются многие интересные утверждения*, которые не всегда выполняются для графов в общем случае.
- Применительно к деревьям многие доказательства и рассуждения оказываются намного проще. Выдвигая какие-то гипотезы при решении задач теории графов, целесообразно сначала их проверять на деревьях.
- Деревья являются *самым распространённым классом графов, применяемых в программировании.*

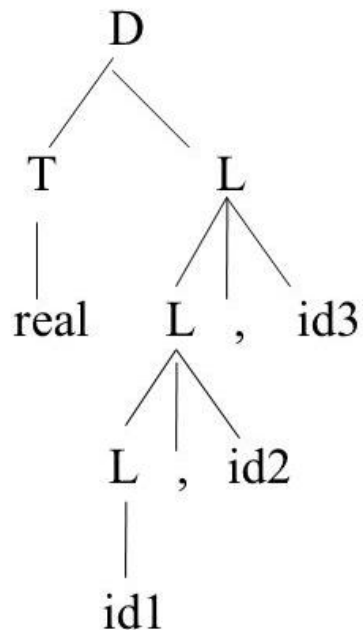
Примеры деревьев



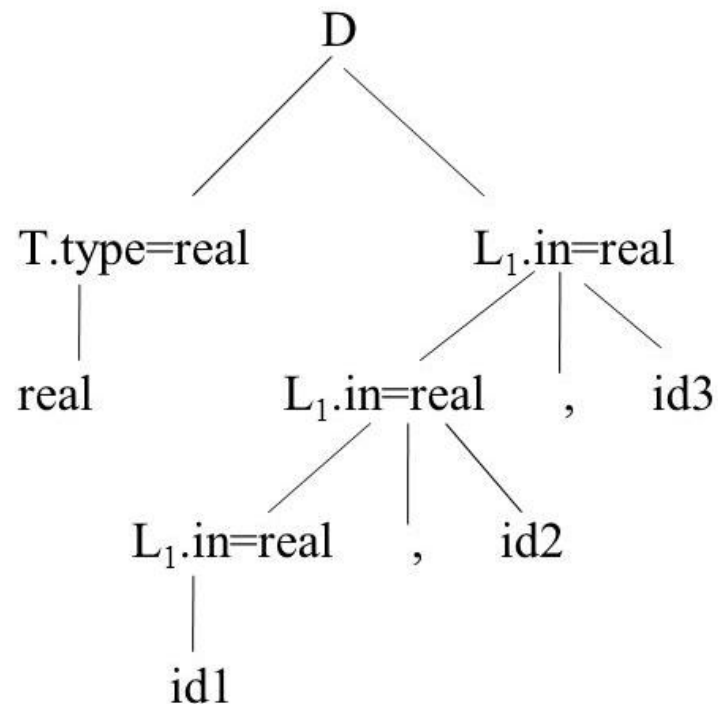
Примеры деревьев

Input: real p, q, r

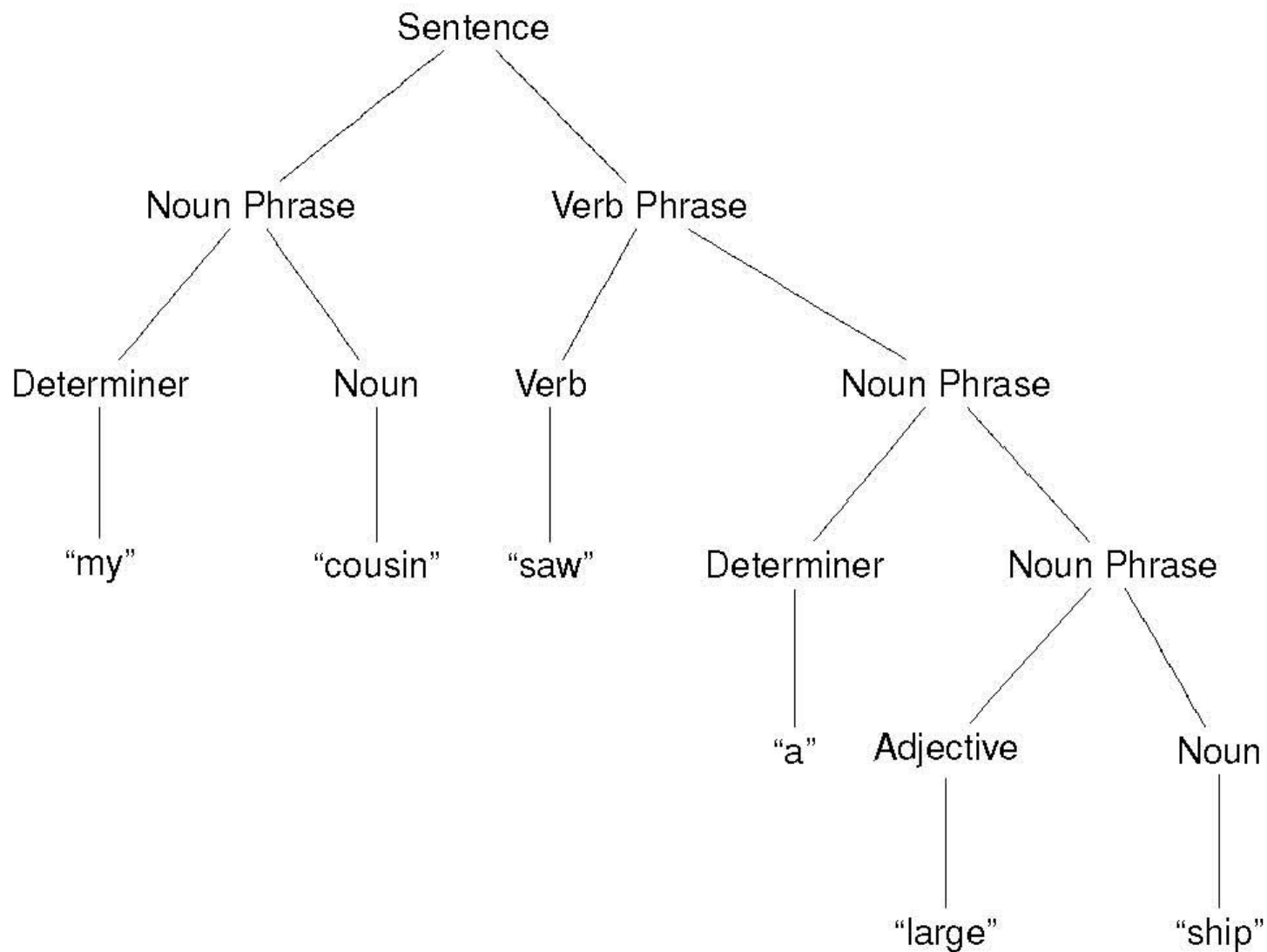
parse tree



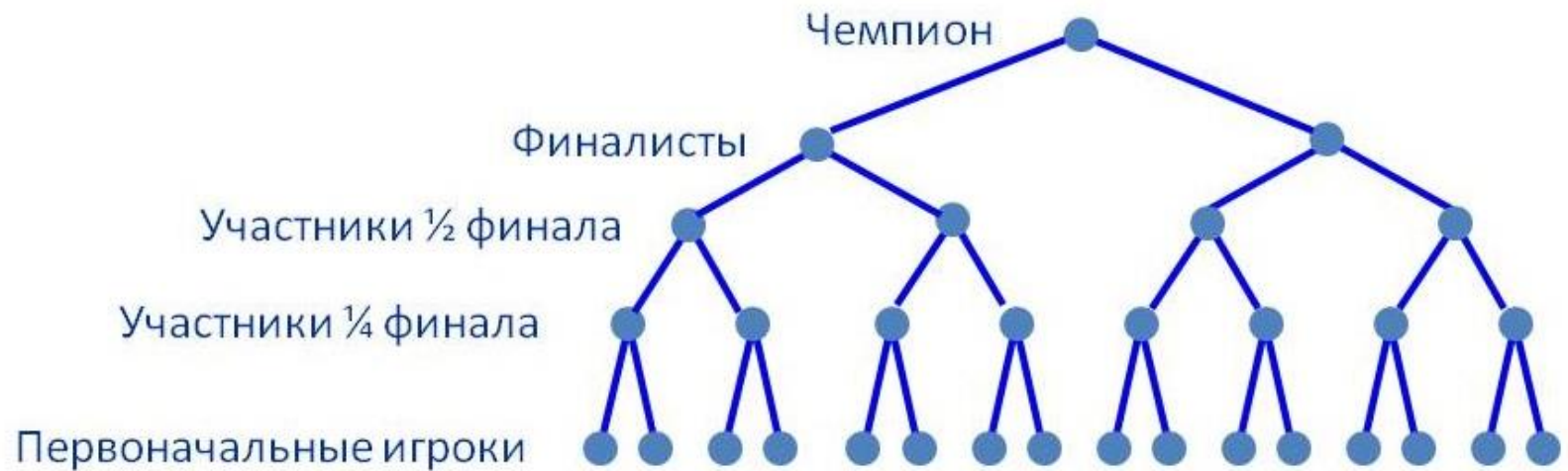
annotated parse tree



Примеры деревьев

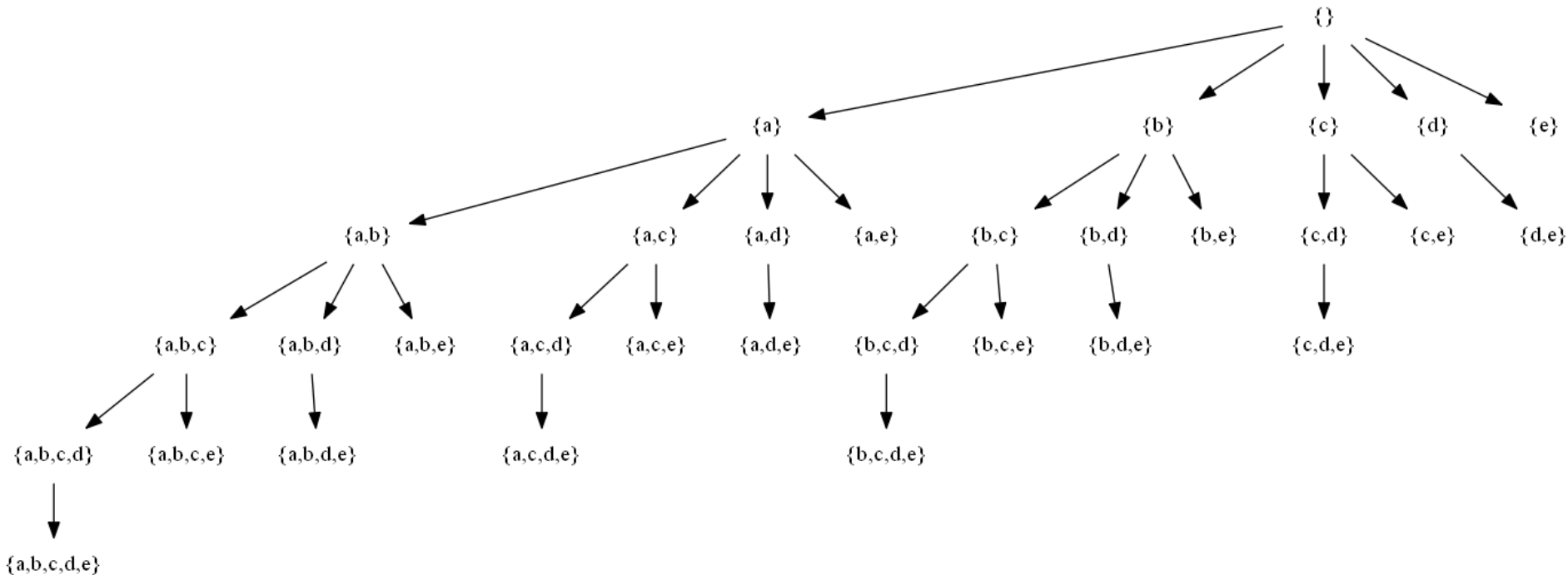


Примеры деревьев



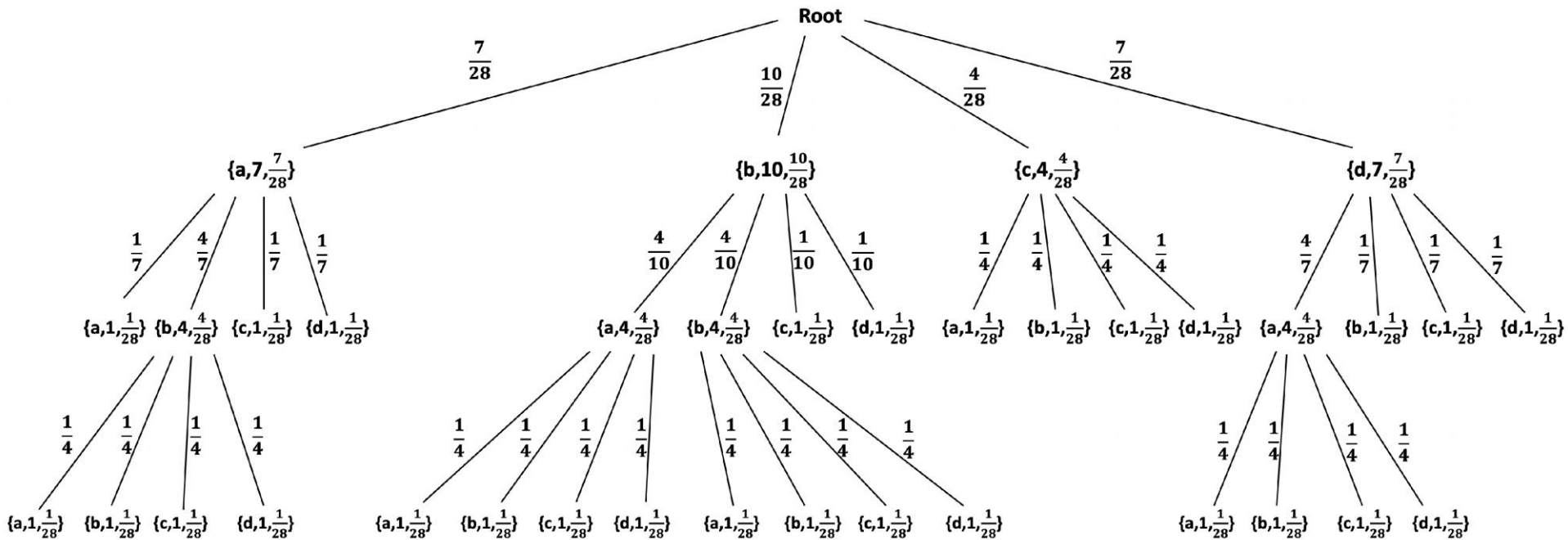
Примеры деревьев

- $\Sigma = \{a,b,c,d,e\}$
- Какое **отношение** здесь можно определить?!

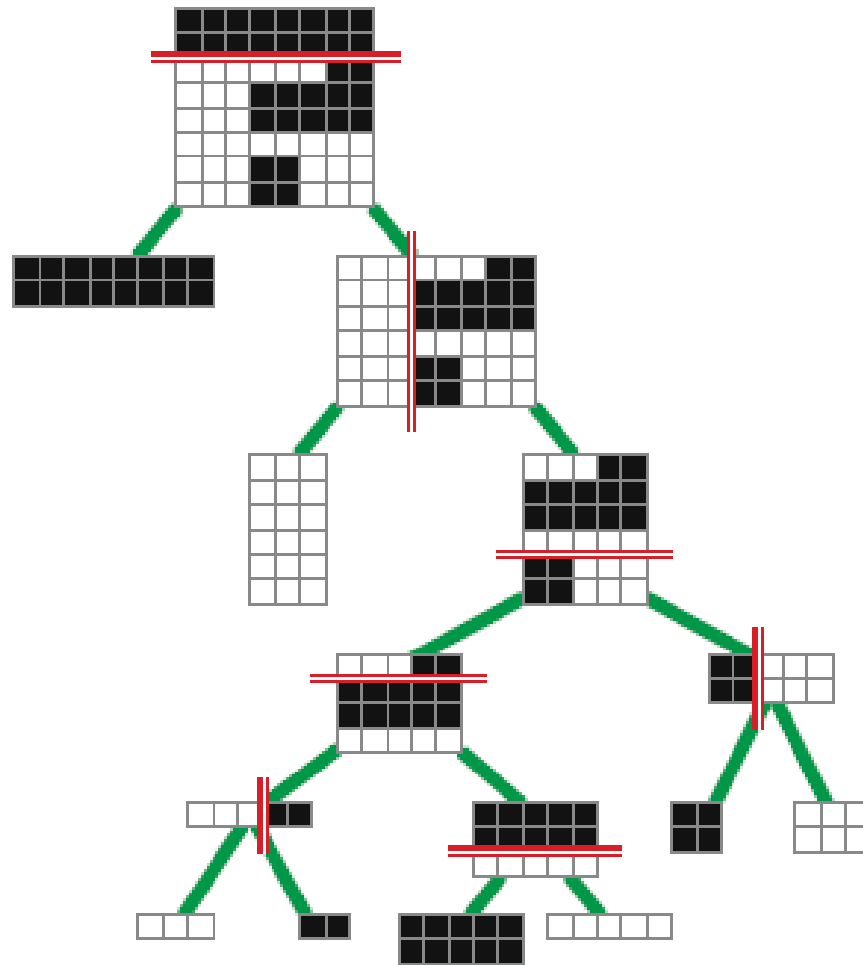


Примеры деревьев

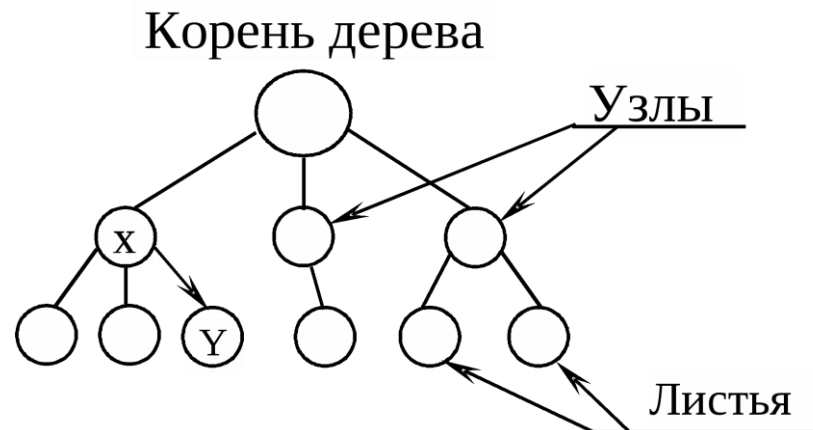
- $\Sigma = \{a,b,c,d\}$
- Какое **отношение (правило)** здесь можно определить?!



Гильотинное подразбиение карты пикселей



Определение дерева

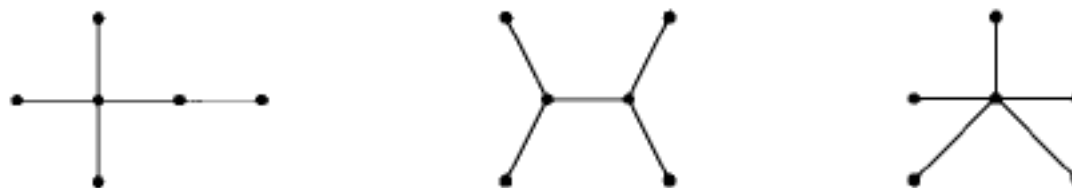
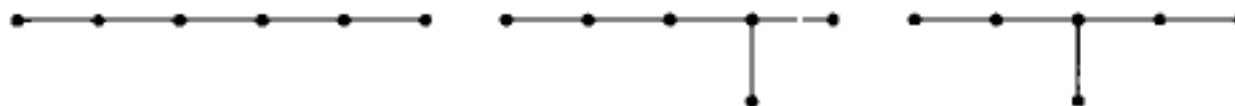


- Граф без циклов называется **ациклическим** или лесом.
- Связный ациклический граф называется **(свободным) деревом**.
- Таким образом, *компонентами связности леса* являются **деревья**.
- Прилагательное «свободное» употребляется в том случае, когда нужно подчеркнуть отличие деревьев от других объектов, родственных деревьям: ориентированных деревьев, упорядоченных деревьев и т. д.
- В связном графе G выполняется неравенство $q(G) \geq p(G) - 1$.
- Граф G (не обязательно связный!), в котором $q(G) = p(G) - 1$, называется **древочисленным**.

Определение дерева

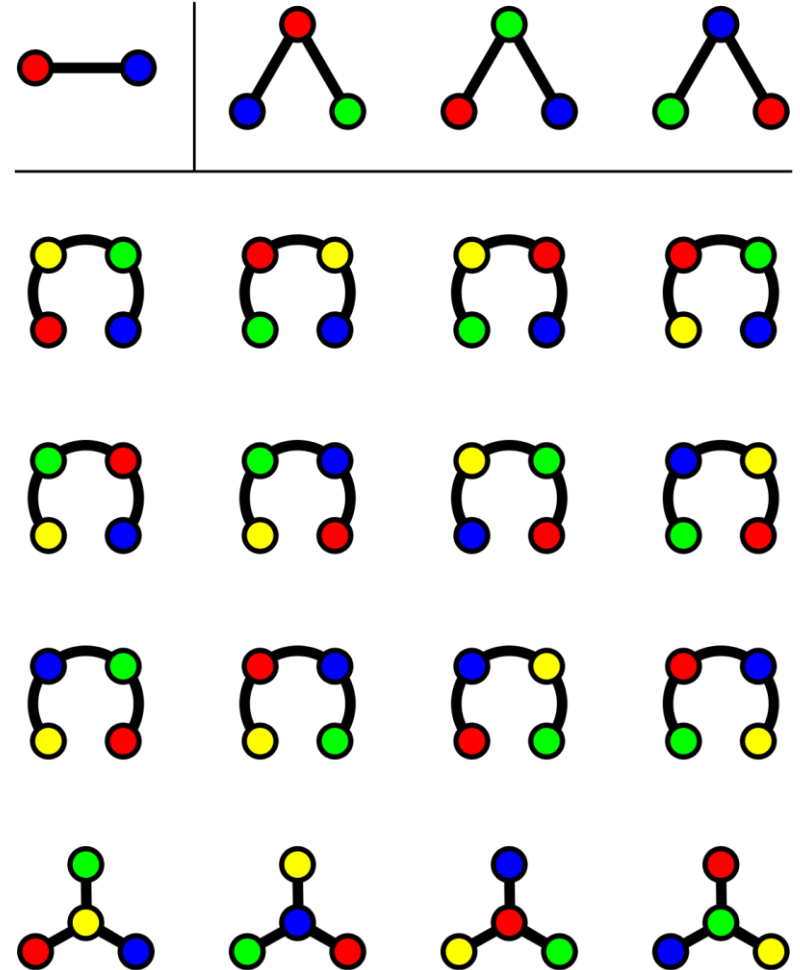
- В ациклическом графе G $z(G) = 0$.
- Пусть u, v — несмежные вершины графа G , $x = (u, v)$ не принадлежит E .
- Если граф $G + x$ имеет ровно один простой цикл, $z(G + x) = 1$, то граф G (не обязательно ациклический!) называется **субциклическим**.
- Графы $K_3 \cup K_1$ и $K_3 \cup K_2$ являются *древочисленными* и *субциклическими* и в то же время не являются ни связными, ни ациклическими.

(Свободных) деревья с 5 и 6 вершинами



Теорема Кэли

- **Теорема Кэли.** Число различных деревьев, которые можно построить на $n \geq 2$ различных вершинах, равно n^{n-2} .



Основные свойства деревьев

Пусть $G(V, E)$ — граф с p вершинами, q ребрами, k компонентами связности и z простыми циклами. Пусть x — ребро, соединяющее любую пару несмежных вершин в G . Тогда следующие утверждения эквивалентны:

1. G — дерево, то есть связный граф без циклов,

$$k(G) = 1 \ \& \ z(G) = 0.$$

2. Любые две вершины соединены в G единственной простой цепью,

$$\forall u, v \ (|P(u, v)| = 1).$$

3. G — связный граф, и любое ребро есть мост,

$$k(G) = 1 \ \& \ \forall e \in E \ (k(G - e) > 1).$$

4. G — связный и древочисленный,

$$k(G) = 1 \ \& \ q(G) = p(G) - 1.$$

Основные свойства деревьев

5. G — ациклический и древочисленный,

$$z(G) = 0 \ \& \ q(G) = p(G) - 1.$$

6. G — ациклический и субциклический,

$$z(G) = 0 \ \& \ z(G + x) = 1.$$

7. G — связный, субциклический и неполный,

$$k(G) = 1 \ \& \ G \neq K_p \ \& \ p \geq 3 \ \& \ z(G + x) = 1.$$

8. G — древочисленный и субциклический (за двумя исключениями),

$$q(G) = p(G) - 1 \ \& \ G \neq K_1 \cup K_3 \ \& \ G \neq K_2 \cup K_3 \ \& \ z(G + x) = 1.$$

СЛЕДСТВИЕ 1. В любом нетривиальном дереве имеются по крайней мере две висячие вершины.

Например, концы диаметра.

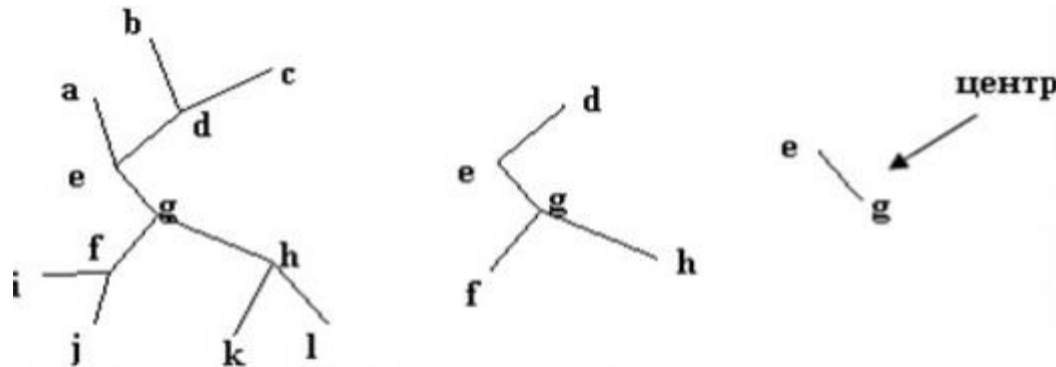
СЛЕДСТВИЕ 2. Каждая не висячая вершина свободного дерева является точкой сочленения.

СЛЕДСТВИЕ 3. Если в связном графе нет висячих вершин, то в нём есть цикл.

Центр дерева

- Центр свободного дерева состоит из одной вершины или из двух смежных вершин:

$$(z(G) = 0 \ \& \ k(G) = 1) \implies (C(G) = K_1 \vee C(G) = K_2).$$



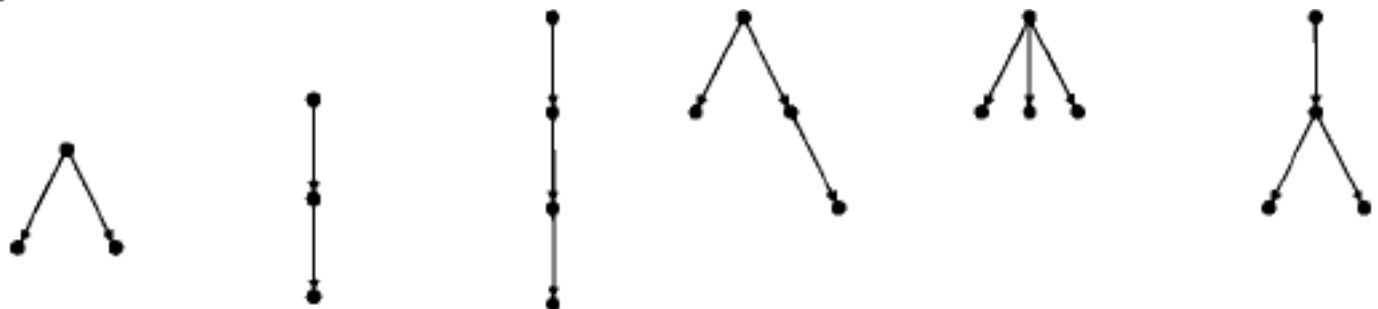
Ориентированные деревья

Ориентированным деревом (или ордеревом, или корневым деревом) называется орграф со следующими свойствами:

1. Существует единственный узел r , полустепень захода которого равна 0, $d^+(r) = 0$. Он называется *корнем* ордерова.
2. Полустепень захода всех остальных узлов равна 1, $\forall v \in V - r \ (d^+(v) = 1)$.
3. Каждый узел достижим из корня, $\forall v \in V - r \ (\exists \langle r, v \rangle)$.

ТЕОРЕМА *Ордерено обладает следующими свойствами:*

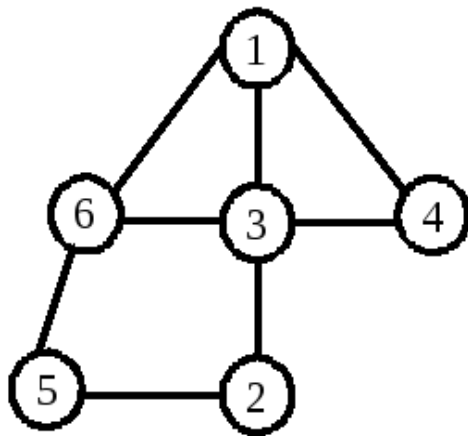
1. $q = p - 1$.
2. Если в ордереве забыть ориентацию дуг, то получится свободное дерево.
3. В ордереве нет контуров.
4. Для каждого узла существует единственный путь, ведущий в этот узел из корня.
5. Подграф, определяемый множеством узлов, достижимых из узла v , является ордеревом с корнем v (это ордерено называется поддеревом узла v).
6. Если в свободном дереве любую вершину назначить корнем, то получится ордерено.



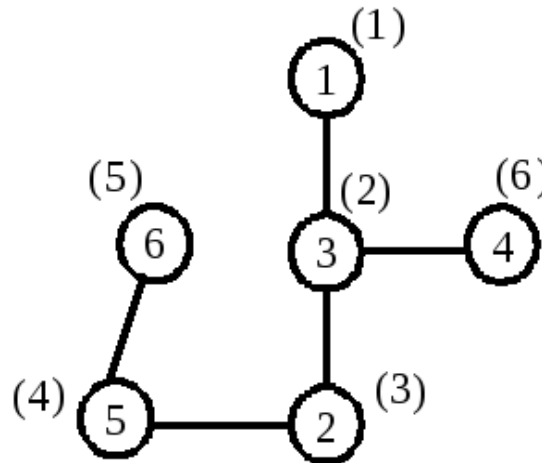
Комментарии

Алгоритм поиска в глубину строит ордеререво с корнем в начальном узле.

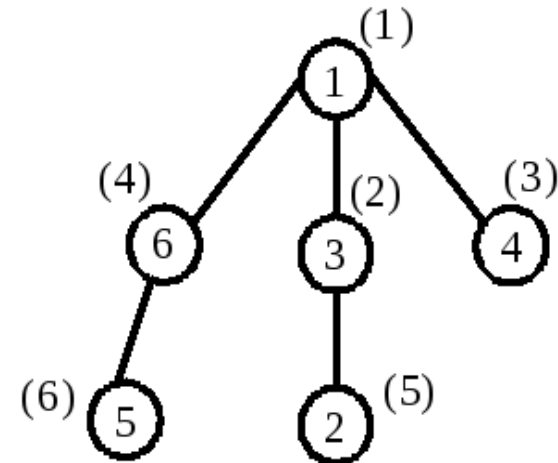
- Каждую вершину в свободном дереве с p вершинами можно назначить корнем и получить ордеререво. Некоторые из полученных ордеревьев могут оказаться *изоморфными*.
- Свободное дерево определяет не более p различных ориентированных деревьев.
- Таким образом, общее число различных ордеревьев с p узлами не более чем в p раз превосходит общее число различных свободных деревьев с p вершинами.



Граф



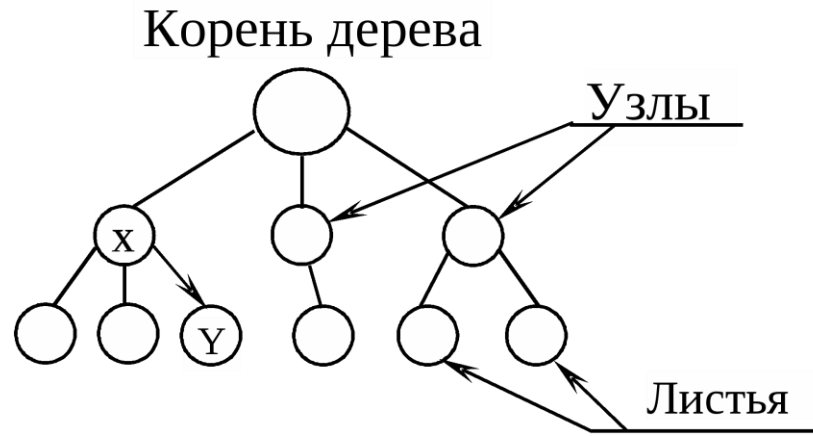
Поиск в глубину



Поиск в ширину

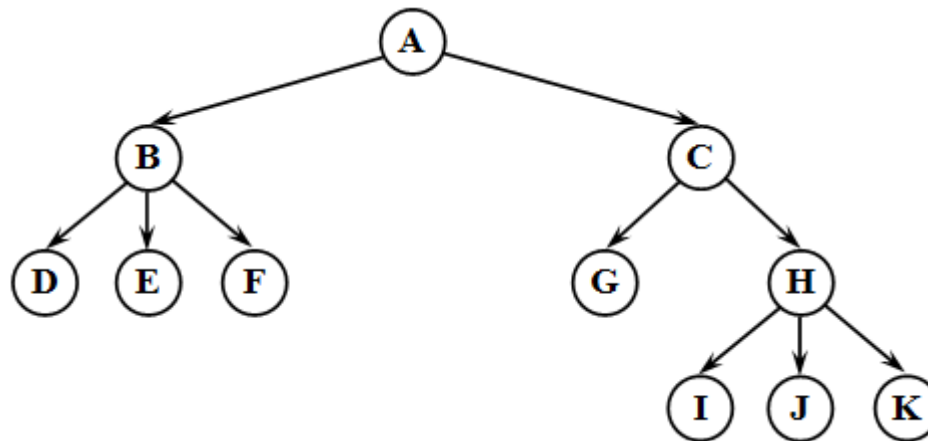
Дополнительные определения

- Концевая вершина ордеререва называется листом. Множество листьев называется **кроной**.
- Путь из корня в лист называется **ветвью**.
- Длина наибольшей ветви ордеререва называется его **высотой**. Уровень узла ордеререва — это расстояние от корня до узла.
- Сам корень имеет **уровень 0**.
- Узлы одного уровня образуют **ярус** ордеререва.

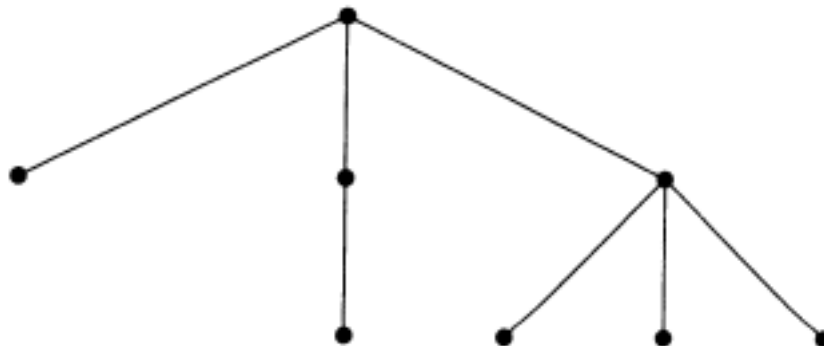
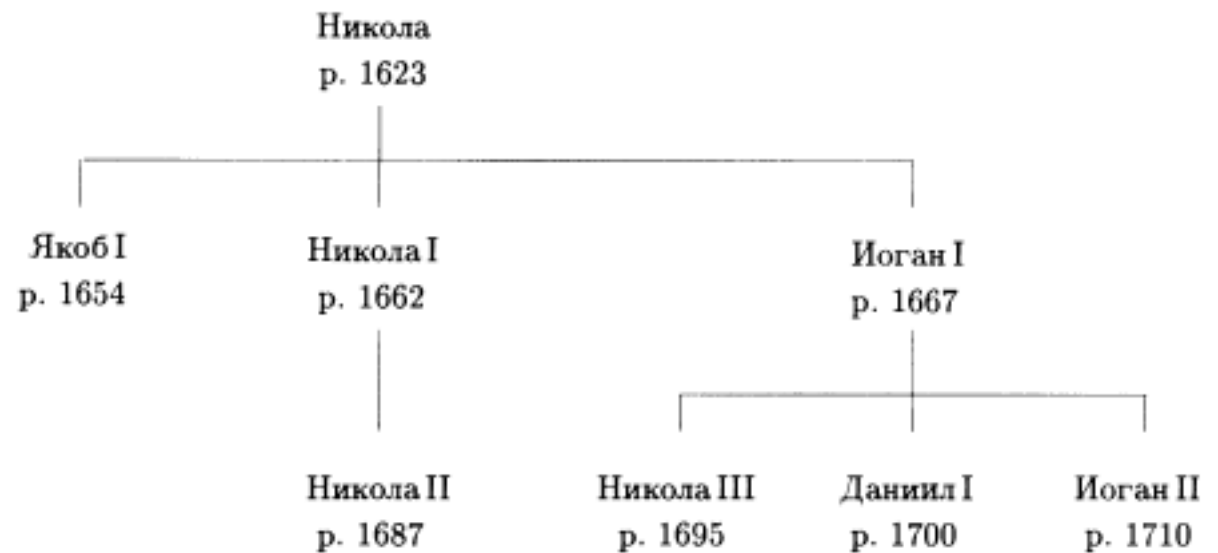


Генеалогическая терминология

- Наряду с «растительной» применяется еще и «генеалогическая» терминология.
- Узлы, достижимые из узла s , называются потомками узла s (потомки одного узла образуют поддерево).
- Если узел h является потомком узла s , то узел s называется предком узла h .
- Если в дереве существует дуга (s, h) , то узел s называется *отцом* (или родителем) узла h , а узел h называется *сыном* узла s .
- Сыновья одного отца называются *братьями*.

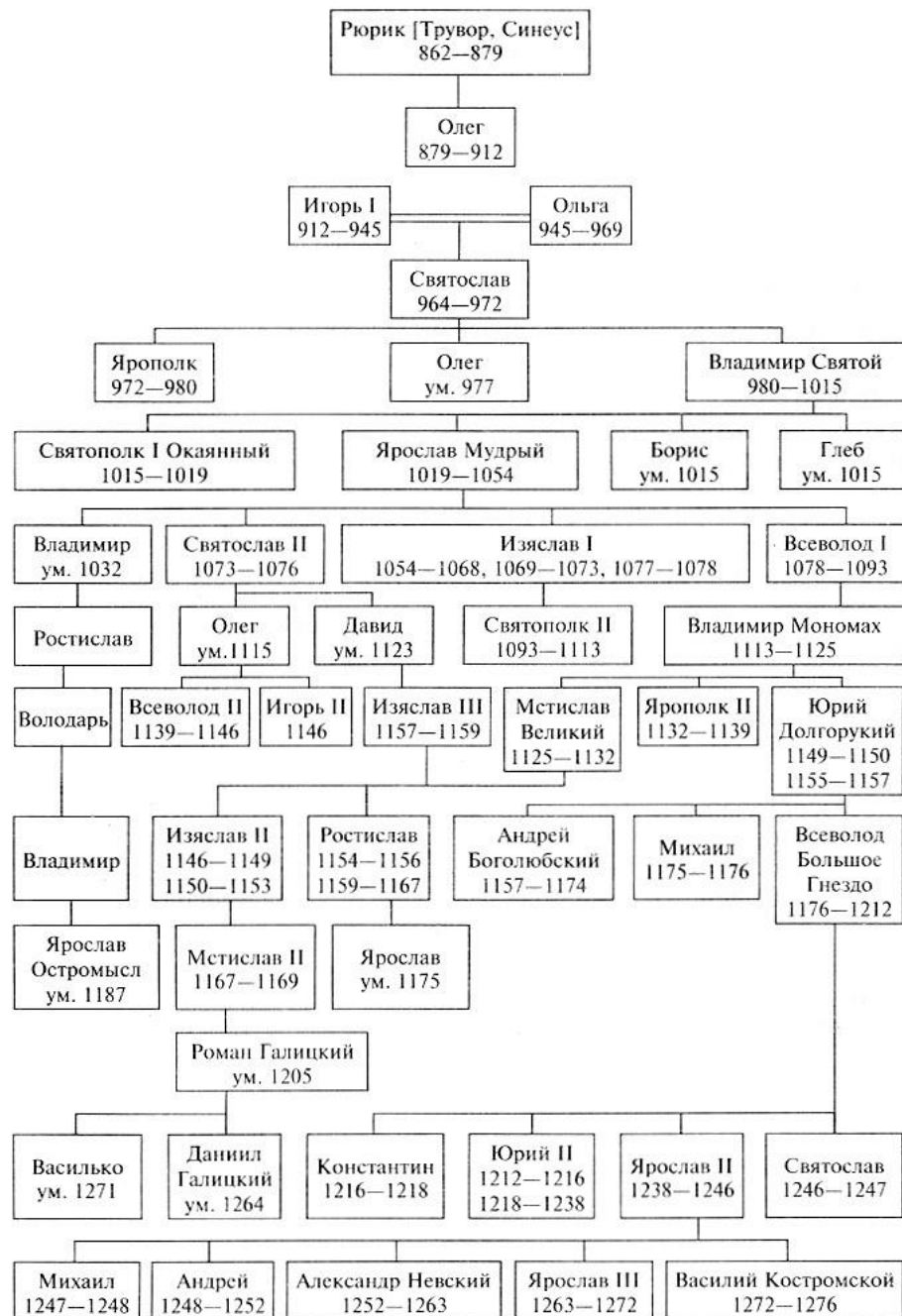


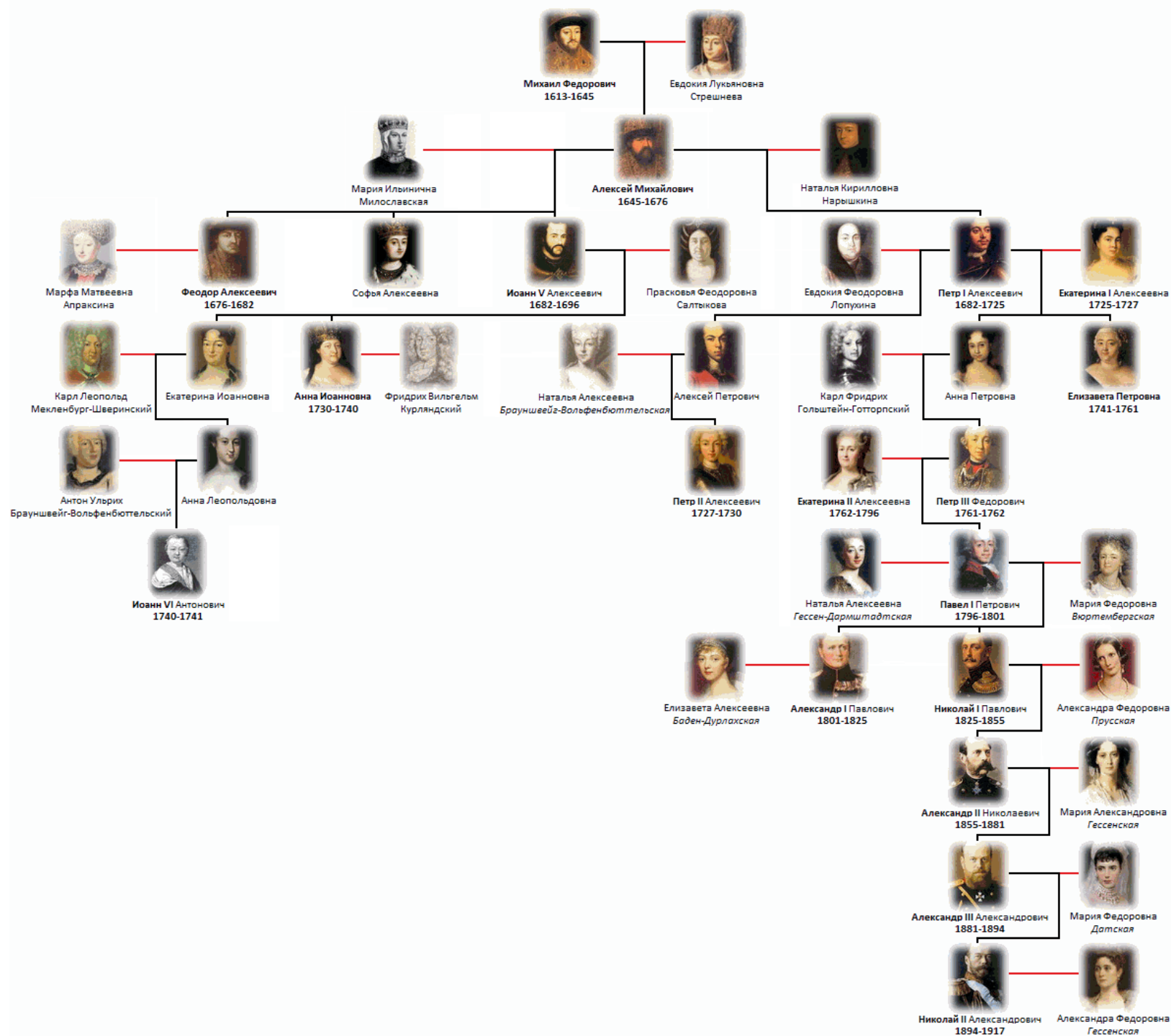
Генеалогическое дерево Бернулли



*Содержит ли оно
информацию?*

Пример





Эквивалентное определение ордера

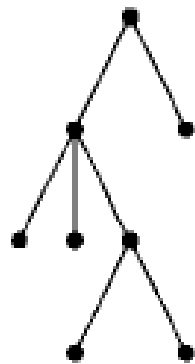
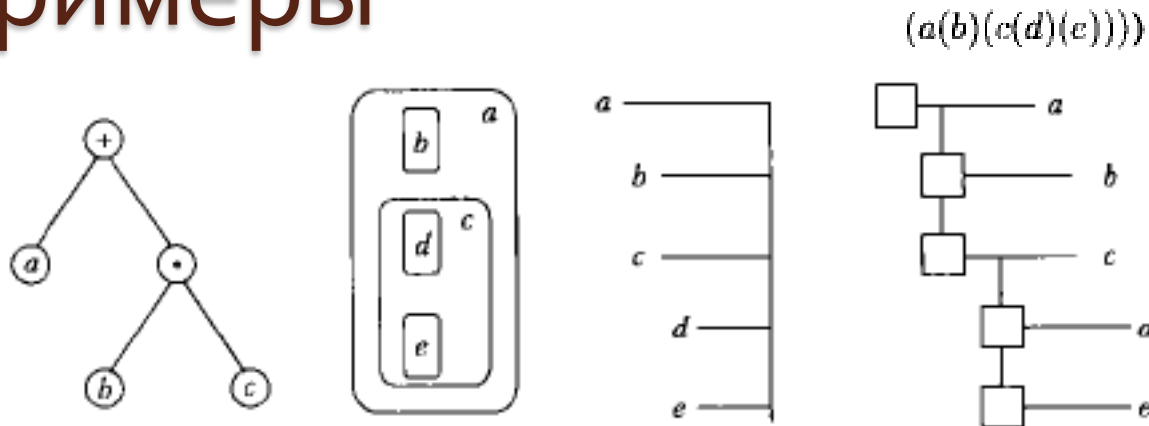
Ордерено T — это непустое конечное множество узлов, на котором определено разбиение, обладающее следующими свойствами:

1. Имеется один выделенный одноэлементный блок $\{r\}$, называемый корнем данного ордера.
2. Остальные узлы (исключая корень) содержатся в k ($k \geq 0$) блоках $T_1, \dots, T_k$, каждый из которых, в свою очередь, является ордереном и называется поддеревом.

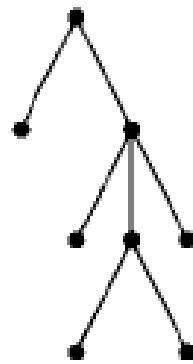
$$T \stackrel{\text{Def}}{=} \{\{r\}, T_1, \dots, T_k\}.$$

Если относительный порядок поддеревьев $T_1, \dots, T_k$ в эквивалентном определении ордера фиксирован, то ордерено называется *упорядоченным*.

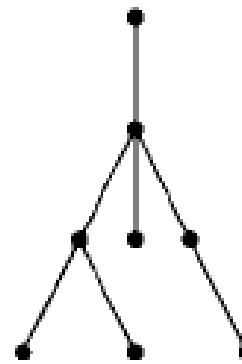
Примеры



(1)



(2)



(3)

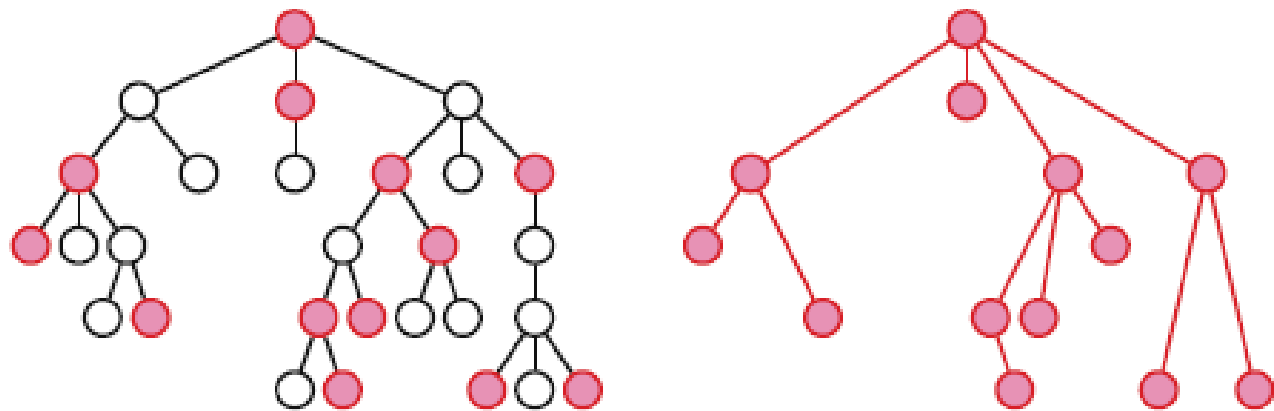
Как свободные деревья – изоморфны.

Как упорядоченные деревья – все различны.

Как ориентированные деревья – $1=2$, но $2 \neq 3$.

Миноры с корнями (rooted minor)

- Естественное обобщение подпоследовательностей.
- **Минор с корнем** дерева с корнем T – любое дерево, полученное посредством стягивания одного или нескольких ребер.
- При стягивании ребра $u-v$, где u – родитель v , потомок узла v становится новым потомком узла u , затем узел v удаляется.



Бинарные деревья

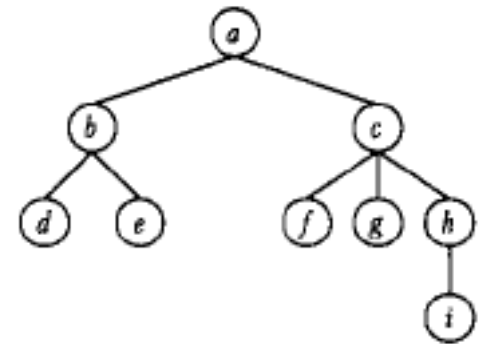
Бинарное (или двоичное) дерево — это непустое конечное множество узлов, на котором определена структура, обладающая следующими свойствами:

- 1. Имеется один выделенный узел $г$, называемый корнем данного бинарного дерева.
- 2. Остальные узлы (исключая корень) содержатся в двух непересекающихся множествах (поддеревах) — левом и правом, каждое из которых, в свою очередь, либо пусто, либо является бинарным деревом.

Бинарное дерево не является упорядоченным ордеревом. Даже если у некоторого узла бинарного дерева имеется только одно непустое поддерево, то всё равно известно, какое именно это поддерево: левое или правое.



m-ичное дерево



- Понятие двоичного дерева допускает обобщение, m -ичным деревом называется непустое конечное множество узлов, которое состоит из корня и m непересекающихся подмножеств, имеющих номера $1, \dots, m$, каждое из которых в свою очередь, либо *пусто*, либо является m -ичным деревом.
- Большая часть утверждений и алгоритмов для двоичных деревьев может быть сравнительно легко распространена и на m -ичные деревья (с соответствующими модификациями). Поэтому хотя *на практике* m -ичные деревья и используются достаточно часто, мы ограничиваемся только двоичными деревьями.

Представление свободных деревьев – код Прюфера

Вершины дерева $T(V, E)$ занумерованы числами из интервала $1..p$.
Построим последовательность A : array $[1..p-1]$ of $1..p$

Вход: Дерево $T(V, E)$ в любом представлении, вершины дерева занумерованы числами $1..p$ произвольным образом.

Выход: Массив A : array $[1..p-1]$ of $1..p$ – код Прюфера дерева T .

```
for i from 1 to p-1 do
  v := min { k ∈ V | d(k) = 1 } { выбираем вершину v – висячую вершину с
    наименьшим номером }
  A[i] := Γ(v) { запосим в код номер единственной вершины, смежной с v }
  V := V - v { удаляем вершину v из дерева }
end for
```

Код Прюфера — наиболее экономное по памяти представление дерева. Его можно немного улучшить, если заметить, что существует всего одно дерево с двумя вершинами — K_2 , а потому информацию о последнем ребре можно не хранить, она восстанавливается однозначно.

Распаковка кода Прюфера свободного дерева

Вход: Массив A : **array** $[1..p - 1]$ **of** $1..p$ — код Прюфера дерева T .

Выход: Дерево $T(V, E)$, заданное множеством рёбер E , вершины дерева пронумерованы числами $1..p$.

$E := \emptyset$ { в начале множество рёбер пусто }

$B := 1..p$ { множество неиспользованных номеров вершин }

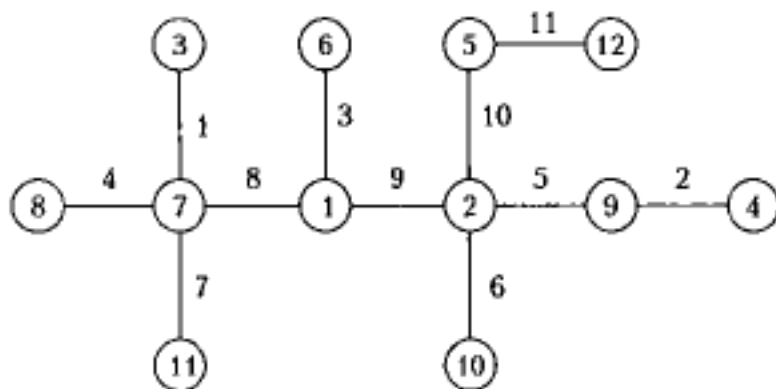
for i **from** 1 **to** $p - 1$ **do**

$v := \min \{k \in B \mid \forall j \geq i (k \neq A[j])\}$ { выбираем вершину v — неиспользованную вершину с наименьшим номером, который не встречается в остатке кода Прюфера }

$E := E + (v, A[i])$ { добавляем ребро $(v, A[i])$ }

$B := B - v$ { удаляем вершину v из списка неиспользованных }

end for



7,9,1,7,2,2,7,1,2,5,12

Построение кода упорядоченного ордерова

Вход: Нетривиальное упорядоченное ордеро T , заданное списками смежности $\Gamma : \text{array } [1..p] \text{ of } \uparrow N$, где $N = \text{record } v : 1..p; n : \uparrow N \text{ end record}$, причём узлы перенумерованы в прямом порядке.

Выход: Массив $C : \text{array } [1..2q] \text{ of } 0..1$, являющийся кодом ордерова T .

$i := 0$ { количество заполненных разрядов кода }

TraverseTree(1) { корневой узел имеет номер 1 }

Вход: Число $n : 1..p$ — номер текущего узла.

Выход: Заполнение двух разрядов кода C .

$i := i + 1; C[i] := 1$ { отмечаем вход в узел }

$d := \Gamma[n]$ { указатель на первый элемент списка сыновей }

while $d \neq \text{nil}$ **do**

TraverseTree($d.v$) { построение кода поддерова очередного сына }

$d := d.n$ { выбор следующего сына }

end while

$i := i + 1; C[i] := 0$ { отмечаем выход из узла }

Восстановление упорядоченного ордерова по коду

Вход: Массив $C : \text{array } [1..2q] \text{ of } 0..1$ — код упорядоченного ордерова T .

Выход: Массив $\Gamma : \text{array } [1..p] \text{ of } \uparrow N$, где $N = \text{record } v : 1..p; n : \uparrow N$
end record — списки смежности упорядоченного ордерова T .

$p := 1$ { счётчик узлов }

$\Gamma[1] := \text{nil}$ { корень }

$n := 1$ { текущий узел }

for i **from** 1 **to** $2q$ **do**

if $C[i] = 1$ **then**

$p := p + 1$ { номер нового узла }

$\Gamma[p] := \text{nil}$ { новый узел пока листовой }

$d := \text{NewNode}(p, \text{nil})$ { дуга от текущего узла к новому узлу }

 Append($\Gamma[n], d$) { добавить узел в список смежности }

$n \rightarrow S$ { положить номер текущего узла на стек }

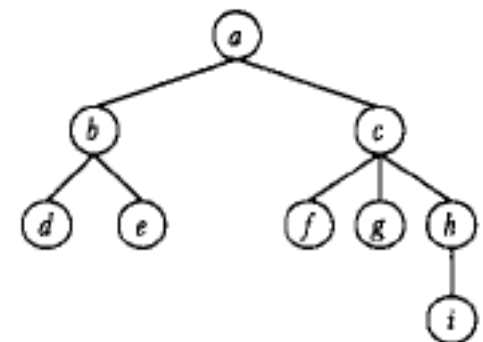
$n := p$ { перейти в новый узел }

else

$n \leftarrow S$ { снять со стека и перейти в новый узел }

end if

end for



1101001101011000

Проверка правильности скобочной структуры

Вход: Строка s : **array** $[1..n]$ **of char**, возможно, содержащая скобки «(» и «)».

Выход: Число 0, если скобочная структура правильна, или число в диапазоне $1..(n + 1)$, указывающее на позицию в строке, где скобочная структура нарушена.

$p := 0$ { число прочитанных открывающих минус число закрывающих скобок }

for i **from** 1 **to** n **do**

if $s[i] = "("$ **then**

$p := p + 1$ { прочли открывающую скобку }

end if

if $s[i] = ")"$ **then**

$p := p - 1$ { прочли закрывающую скобку }

end if

if $p < 0$ **then**

return i { лишняя закрывающая скобка }

end if

end for

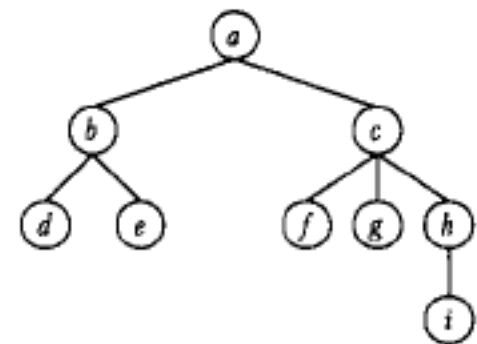
if $p = 0$ **then**

return 0 { скобочная структура правильна }

else

return $n + 1$ { не хватает закрывающих скобок }

end if



$a(b(d)(e))(c(f)(g)(h(i)))$

Представление бинарных деревьев

- Всякое свободное дерево можно **ориентировать**, назначив один из узлов корнем. Всякое ордеререво можно **произвольно упорядочить**. Для потомков одного узла (братьев) упорядоченного ордеререва определено отношение старше-младше (левее-правее). Всякое упорядоченное дерево можно представить бинарным деревом, например, проводя правую связь к старшему брату, а левую — к младшему сыну.
- Таким образом, достаточно рассмотреть *представление в программе* бинарных деревьев. Это наблюдение объясняет, почему представлению бинарных деревьев в программах традиционно уделяется особое внимание при программировании.
- Из данного представления следует, что множество *бинарных деревьев* **взаимооднозначно соответствует** множеству *упорядоченных лесов* упорядоченных ордеререьев: одиночному упорядоченному ордеререву всегда соответствует бинарное дерево, у которого правая связь корня пуста, а упорядоченному лесу — бинарное дерево, у которого правая связь корня не пуста.

Представление бинарных деревьев

1. **Списочные структуры:** каждый узел представляется записью типа N , содержащей два поля (l и r) с указателями на левый и правый узлы и еще одно поле i для хранения указателя на информацию об узле. Дерево представляется указателем на корень. Тип N обычно определяется следующим образом: $N = \text{record } i : \text{info}; l, r : \uparrow N \text{ end record}$, где тип info считается заданным. Для этого представления $n(p) = 3p$.
2. **Упакованные массивы:** все узлы располагаются в массиве, так что все узлы поддерева данного узла располагаются вслед за этим узлом. Вместе с каждым узлом хранится индекс узла, который является первым узлом правого поддерева данного узла. Дерево T обычно определяется следующим образом: $T : \text{array } [1..p] \text{ of record } i : \text{info}, k : 1..p \text{ end record}$, где тип info считается заданным. Для этого представления $n(p) = 2p$.
3. **Польская запись:** аналогично, но вместо связей фиксируется «размеченная степень» каждого узла (например, 0 означает, что это лист, 1 — есть левая связь, но нет правой, 2 — есть правая связь, но нет левой, 3 — есть обе связи). Дерево T определяется следующим образом: $T : \text{array } [1..p] \text{ of record } i : \text{info}, d : 0..3 \text{ end record}$, где тип info считается заданным. Для этого представления $n(p) = 2p$. Если степень узла известна из информации, хранящейся в самом узле, то можно не хранить и степень. Такой способ представления деревьев называется *польской записью* и обычно используется для представления выражений. В этом случае представление дерева оказывается наиболее компактным: объем памяти $n(p) = p$.

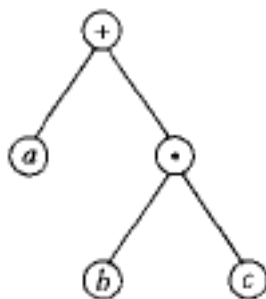
Примеры представления бинарных деревьев



Адрес	<i>i</i>	<i>l</i>	<i>r</i>	<i>i</i>	<i>k</i>	<i>i</i>	<i>d</i>
1	<i>a</i>	2	0	<i>a</i>	0	<i>a</i>	1
2	<i>b</i>	0	0	<i>b</i>	0	<i>b</i>	0

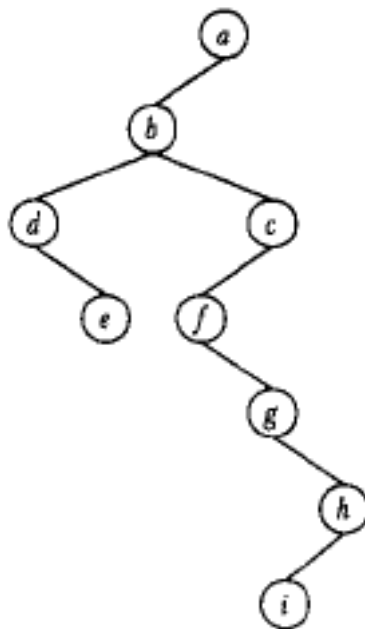


Адрес	<i>i</i>	<i>l</i>	<i>r</i>	<i>i</i>	<i>k</i>	<i>i</i>	<i>d</i>
1	<i>a</i>	0	2	<i>a</i>	2	<i>a</i>	2
2	<i>b</i>	0	0	<i>b</i>	0	<i>b</i>	0



Адрес	<i>i</i>	<i>l</i>	<i>r</i>	<i>i</i>	<i>k</i>	<i>i</i>	<i>d</i>
1	+	2	3	+	3	+	3
2	<i>a</i>	0	0	<i>a</i>	0	<i>a</i>	0
3	*	4	5	*	5	*	3
4	<i>b</i>	0	0	<i>b</i>	0	<i>b</i>	0
5	<i>c</i>	0	0	<i>c</i>	0	<i>c</i>	0

Примеры представления бинарных деревьев



Адрес	<i>i</i>	<i>l</i>	<i>r</i>	<i>i</i>	<i>k</i>	<i>i</i>	<i>d</i>
1	<i>a</i>	2	0	<i>a</i>	0	<i>a</i>	1
2	<i>b</i>	3	5	<i>b</i>	5	<i>b</i>	3
3	<i>d</i>	0	4	<i>d</i>	4	<i>d</i>	2
4	<i>e</i>	0	0	<i>e</i>	0	<i>e</i>	0
5	<i>c</i>	6	0	<i>c</i>	0	<i>c</i>	1
6	<i>f</i>	0	7	<i>f</i>	7	<i>f</i>	2
7	<i>g</i>	0	8	<i>g</i>	8	<i>g</i>	2
8	<i>h</i>	9	0	<i>h</i>	0	<i>h</i>	1
9	<i>i</i>	0	0	<i>i</i>	0	<i>i</i>	0

Обходы бинарных деревьев

Прямой (префиксный, левый) обход:

- попасть в корень,
- обойти левое поддерево,
- обойти правое поддерево.

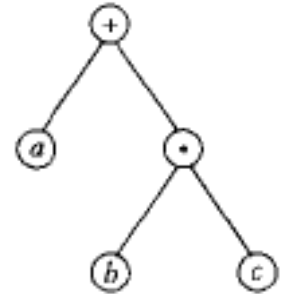
Внутренний (инфиксный, симметричный) обход:

- обойти левое поддерево,
- попасть в корень,
- обойти правое поддерево.

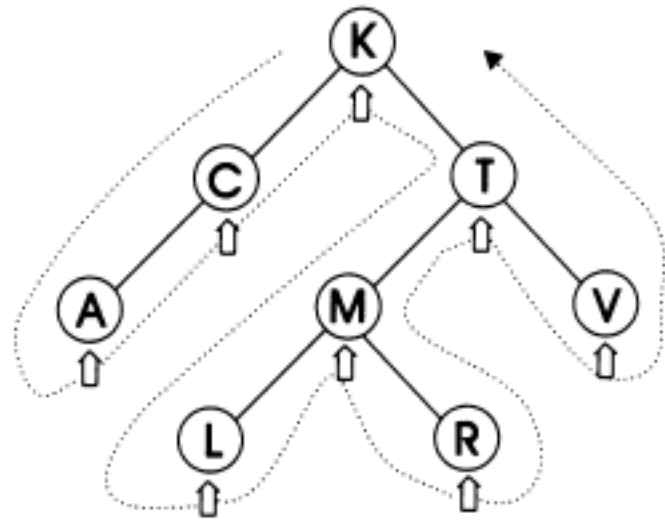
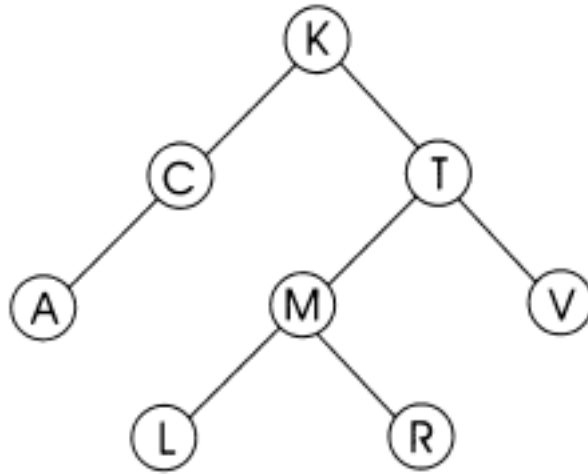
Концевой (постфиксный, правый) обход:

- обойти левое поддерево,
- обойти правое поддерево,
- попасть в корень.

Кроме трёх основных, возможны еще три соответствующих обхода, отличающихся порядком рассмотрения левых и правых поддеревьев. Этим исчерпываются обходы, если в представлении *фиксированы только дуги*, ведущие от отцов к сыновьям.



Пример внутреннего обхода



A, C, K, L, M, R, T, V.

Пример

- Двоичное дерево с 27 узлами, в котором каждый узел помечен не повторяющейся буквой латинского алфавита и знаком &.
- Обход в прямом порядке:

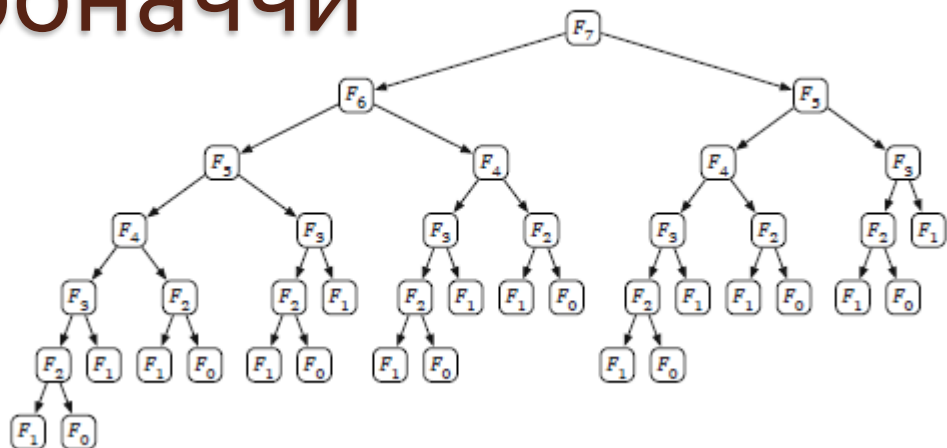
I Q J H L E M V O T S B R G Y Z K C A & F P N U D W X

- Обход в обратном порядке:

H E M L J V Q S G Y R Z B T C P U D N F W & X A K O I

- Изобразить графически двоичное дерево.

Числа Фибоначчи



MEMFIBO(n):

if $n = 0$

return 0

else if $n = 1$

return 1

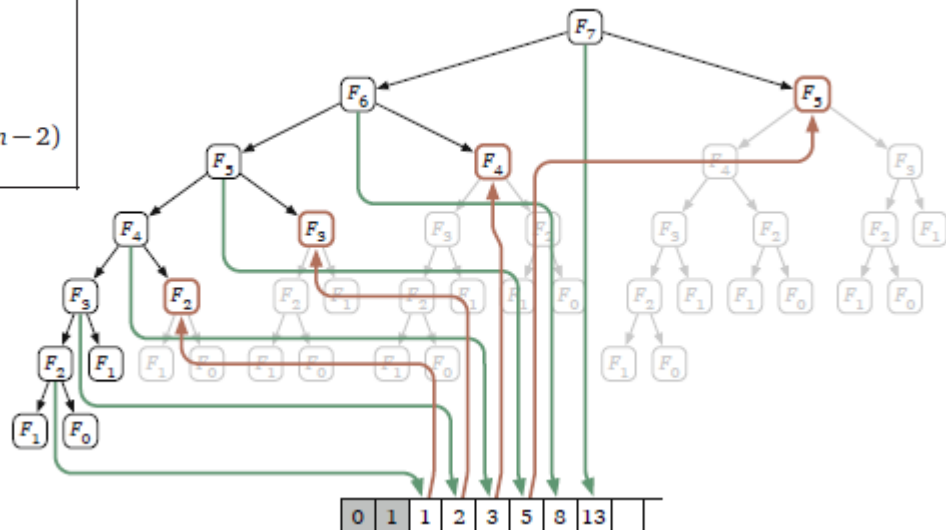
else

if $F[n]$ is undefined

$F[n] \leftarrow \text{MEMFIBO}(n-1) + \text{MEMFIBO}(n-2)$

return $F[n]$

Метод мемоизации
(применение
динамического
программирования)



Польская запись

- Концевой обход дерева выражения $a + b * c$ дает *обратную польскую запись* этого выражения:
 $abc * +.$
- **Польская запись выражений (прямая или обратная)** применяется в некоторых языках программирования непосредственно и используется в качестве внутреннего представления программ во многих **трансляторах** и **интерпретаторах**.
- Причина заключается в том, что такая форма записи допускает очень **эффективную интерпретацию** (вычисление значения) выражений. Например, значение выражения в обратной польской записи может быть вычислено при однократном просмотре выражения слева направо с использованием одного стека. В таких языках, как *Forth* и *PostScript*, обратная польская запись используется как основная.

Алгоритм симметричного обхода бинарного дерева

Вход: бинарное дерево, представленное списочной структурой, r — указатель на корень.

Выход: последовательность узлов бинарного дерева в порядке симметричного обхода.

$T := \emptyset; p := r$ { вначале стек пуст и p указывает на корень дерева }

M : { анализируем узел, на который указывает p }

if $p = \text{nil}$ then

if $T = \emptyset$ then

stop { обход закончен }

end if

$p \leftarrow T$ { левое поддерево обойдено }

yield p { очередной узел при симметричном обходе }

$p := p.r$ { начинаем обход правого поддерева }

else

$p \rightarrow T$ { запоминаем текущий узел... }

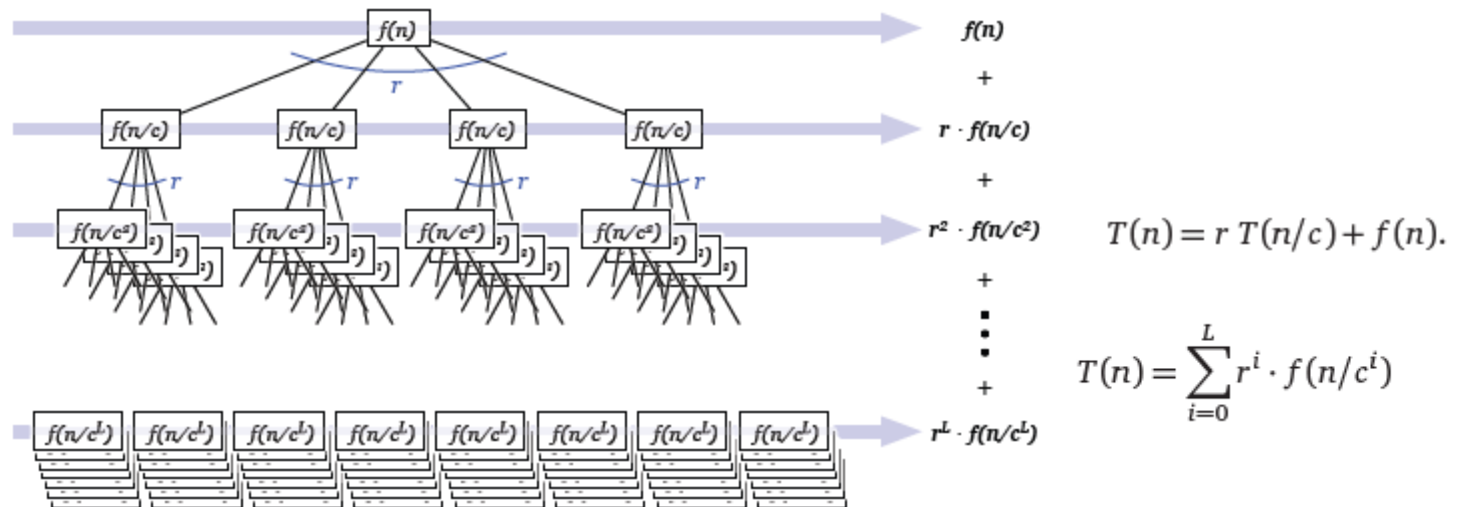
$p := p.l$ { ...и начинаем обход левого поддерева }

end if

goto M

Рекурсивные деревья

- Корневое дерево с одним узлом для каждой рекурсивной подзадачи.
- Значением каждого узла является количество времени для каждой рекурсивной подзадачи, исключая вызовы. Общее время выполнения алгоритма представляет собой сумму значений всех узлов этого дерева.
- Наглядный инструмент общего назначения для решения рекуррентных выражений типа «разделяй и властвуй».



Сортировка слиянием (mergesort)

- Один из самых первых алгоритмов, предназначенных для компьютеров общего назначения с возможностью хранения программ. Разработан Джоном фон Нейманом в 1945 году и подробно описан в работе 1947 года совместно с Германом Голдстейном как одна из первых нечисловых программ для компьютера EDVAC. В действительности они описали нерекурсивный вариант, который теперь называют восходящей сортировкой слиянием (bottom-up mergesort).

- 1. Divide the input array into two subarrays of roughly equal size.
- 2. Recursively mergesort each of the subarrays.
- 3. Merge the newly-sorted subarrays into a single sorted array.

Input:	S	O	R	T	I	N	G	E	X	A	M	P	L	
Divide:	S	O	R	T	I	N		G	E	X	A	M	P	L
Recurse Left:	I	N	O	R	S	T		G	E	X	A	M	P	L
Recurse Right:	I	N	O	R	S	T		A	E	G	L	M	P	X
Merge:	A	E	G	I	L	M	N	O	P	R	S	T	X	

$O(n \log n)$

```
MERGESORT(A[1..n]):
  if n > 1
    m ← ⌊n/2⌋
    MERGESORT(A[1..m])    <<Recurse!>>
    MERGESORT(A[m+1..n])  <<Recurse!>>
    MERGE(A[1..n], m)
```

```
MERGE(A[1..n], m):
  i ← 1; j ← m + 1
  for k ← 1 to n
    if j > n
      B[k] ← A[i]; i ← i + 1
    else if i > m
      B[k] ← A[j]; j ← j + 1
    else if A[i] < A[j]
      B[k] ← A[i]; i ← i + 1
    else
      B[k] ← A[j]; j ← j + 1
  for k ← 1 to n
    A[k] ← B[k]
```

Быстрая сортировка (quicksort)

- Рекурсивный алгоритм сортировки, разработанный Тони Хоаром в 1959 году и впервые опубликованный в 1961 году.
- Основная часть работы заключается в разделении массива на подмассивы меньшего размера перед рекурсией.
 - 1. Choose a *pivot* element from the array.
 - 2. Partition the array into three subarrays containing the elements smaller than the pivot (*опорный*), the pivot element itself, and the elements larger than the pivot.
 - 3. Recursively quicksort the first and last subarrays.

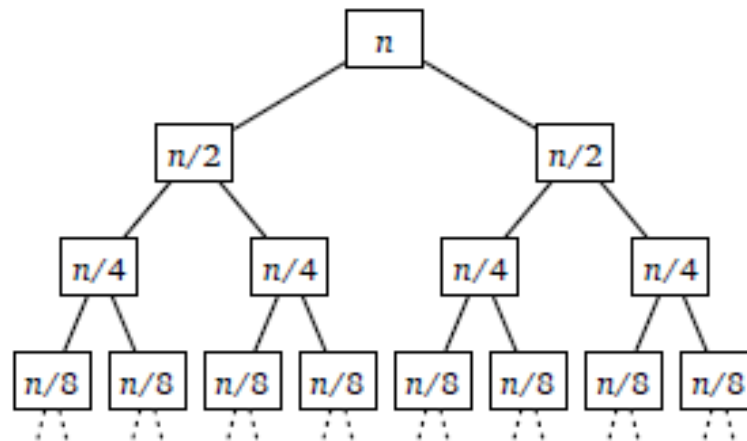
Input:	S	O	R	T	I	N	G	E	X	A	M	P	L
Choose a pivot:	S	O	R	T	I	N	G	E	X	A	M	P	L
Partition:	A	G	O	E	I	N	L	M	P	T	X	S	R
Recurse Left:	A	E	G	I	L	M	N	O	P	T	X	S	R
Recurse Right:	A	E	G	I	L	M	N	O	P	R	S	T	X

«В среднем» $O(n \log n)$
Теоретически $O(n^2)$

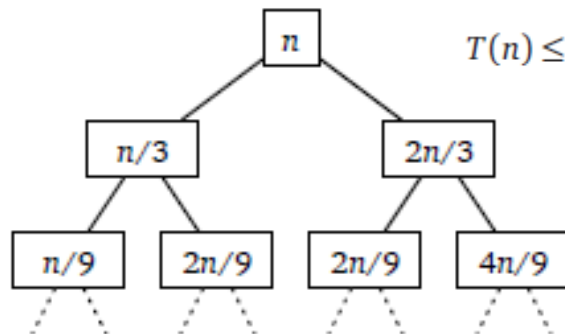
```
QUICKSORT(A[1..n]):  
  if (n > 1)  
    Choose a pivot element A[p]  
    r ← PARTITION(A, p)  
    QUICKSORT(A[1..r - 1])  «Recurse!»  
    QUICKSORT(A[r + 1..n]) «Recurse!»
```

```
PARTITION(A[1..n], p):  
  swap A[p] ↔ A[n]  
  ℓ ← 0      «#items < pivot»  
  for i ← 1 to n - 1  
    if A[i] < A[n]  
      ℓ ← ℓ + 1  
      swap A[ℓ] ↔ A[i]  
  swap A[n] ↔ A[ℓ + 1]  
  return ℓ + 1
```

Рекурсивные деревья для алгоритма сортировки слиянием и быстрой сортировки

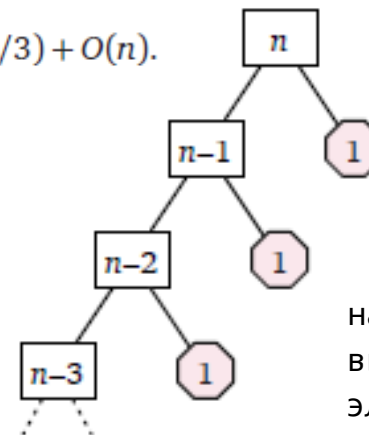


$$T(n) = 2T(n/2) + O(n),$$



правильно выбраны опорные элементы

$$T(n) \leq T(n/3) + T(2n/3) + O(n).$$



наихудший
выбор опорного
элемента

Кратчайший остов

- Пусть $G(V, E)$ — граф. Остовный подграф графа $G(V, E)$ — это подграф, содержащий все вершины. Остовный подграф, являющийся деревом, называется **остовом** или **каркасом**.
- Остов определяется *множеством рёбер*, поскольку вершины остова суть вершины графа.
- Несвязный граф *не имеет остова*. Связный граф может иметь много остовов.
- Если задать длины рёбер, то можно поставить задачу нахождения **кратчайшего остова**.
- Эта задача имеет множество практических интерпретаций. Например, пусть задано множество аэродромов и нужно определить минимальный (по сумме расстояний) набор авиарейсов, который позволил бы перелететь с любого аэродрома на любой другой. Решением этой задачи будет кратчайший остов полного графа расстояний между аэродромами.
- Существует множество различных способов найти некоторый остов графа. Например, алгоритм поиска в глубину строит остов (по рёбрам возврата). Множество кратчайших путей из заданной вершины ко всем остальным также образует остов. Однако этот остов может быть не кратчайшим.

Теорема Кирхгофа

- **Теорема Кирхгофа.** Число остовных деревьев в связном графе G порядка $n \geq 2$ равно алгебраическому дополнению любого элемента матрицы Кирхгофа $B(G)$.
- **Н.В.** Все алгебраические дополнения матрицы Кирхгофа равны между собой.

$$A = \begin{bmatrix} 0 & 1 & 0 & 1 & 0 & 0 \\ 1 & 0 & 1 & 1 & 1 & 1 \\ 0 & 1 & 0 & 0 & 1 & 0 \\ 1 & 1 & 0 & 0 & 1 & 0 \\ 0 & 1 & 1 & 1 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 \end{bmatrix}$$

$$B = \begin{bmatrix} 2 & -1 & 0 & -1 & 0 & 0 \\ -1 & 5 & -1 & -1 & -1 & -1 \\ 0 & -1 & 2 & 0 & -1 & 0 \\ -1 & -1 & 0 & 3 & -1 & 0 \\ 0 & -1 & -1 & -1 & 3 & 0 \\ 0 & -1 & 0 & 0 & 0 & 1 \end{bmatrix}$$

$$\begin{aligned} A_{23} &= (-1)^{2+3} \begin{vmatrix} 2 & -1 & -1 & 0 & 0 \\ 0 & -1 & 0 & -1 & 0 \\ -1 & -1 & 3 & -1 & 0 \\ 0 & -1 & -1 & 3 & 0 \\ 0 & -1 & 0 & 0 & 1 \end{vmatrix} = (-1)^5 \begin{vmatrix} 2 & -1 & -1 & 0 & 0 \\ 0 & -1 & 0 & -1 & 0 \\ -1 & -1 & 3 & -1 & 0 \\ 0 & -1 & -1 & 3 & 0 \\ 0 & 0 & 0 & 0 & 1 \end{vmatrix} = (-1)^{5+5+5} \begin{vmatrix} 2 & -1 & -1 & 0 \\ 0 & -1 & 0 & -1 \\ -1 & -1 & 3 & -1 \\ 0 & -1 & -1 & 3 \end{vmatrix} = \\ &= (-1)^{15} \begin{vmatrix} 2 & -1 & -1 & 0 \\ 0 & 0 & 0 & -1 \\ -1 & 0 & 3 & -1 \\ 0 & -4 & -1 & 3 \end{vmatrix} = (-1)^{16+4+2} \begin{vmatrix} 2 & -1 & -1 \\ -1 & 0 & 3 \\ 0 & -4 & -1 \end{vmatrix} = (-1)^{22} \begin{vmatrix} 2 & -1 & 5 \\ -1 & 0 & 0 \\ 0 & -4 & -1 \end{vmatrix} = (-1)^{28} \begin{vmatrix} -1 & 5 \\ -4 & -1 \end{vmatrix} = 1 + 20 = 21 \end{aligned}$$

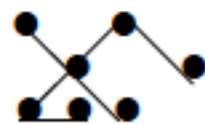
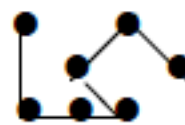
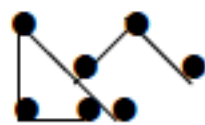
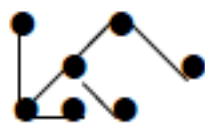
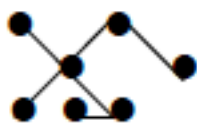
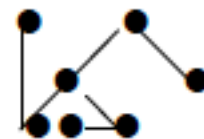
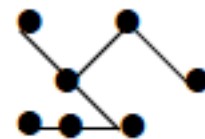
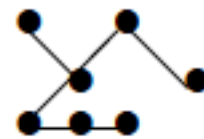
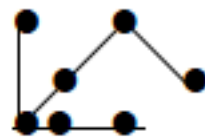
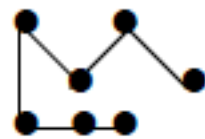
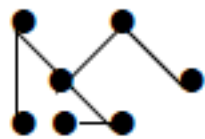
$$B = \begin{matrix} & \begin{matrix} x_1 & x_2 & x_3 & x_4 & x_5 & x_6 & x_7 \end{matrix} \\ \begin{matrix} x_1 \\ x_2 \\ x_3 \\ x_4 \\ x_5 \\ x_6 \\ x_7 \end{matrix} & \begin{pmatrix} 2 & -1 & 0 & 0 & -1 & 0 & 0 \\ -1 & 3 & -1 & 0 & -1 & 0 & 0 \\ 0 & -1 & 2 & -1 & 0 & 0 & 0 \\ 0 & 0 & -1 & 2 & -1 & 0 & 0 \\ -1 & -1 & 0 & -1 & 4 & -1 & 0 \\ 0 & 0 & 0 & 0 & -1 & 2 & -1 \\ 0 & 0 & 0 & 0 & 0 & -1 & 1 \end{pmatrix} \end{matrix}$$

$$A_{11} = \begin{vmatrix} 3 & -1 & 0 & -1 & 0 & 0 \\ -1 & 2 & -1 & 0 & 0 & 0 \\ 0 & -1 & 2 & -1 & 0 & 0 \\ -1 & 0 & -1 & 4 & -1 & 0 \\ 0 & 0 & 0 & -1 & 2 & -1 \\ 0 & 0 & 0 & 0 & -1 & 1 \end{vmatrix} =$$

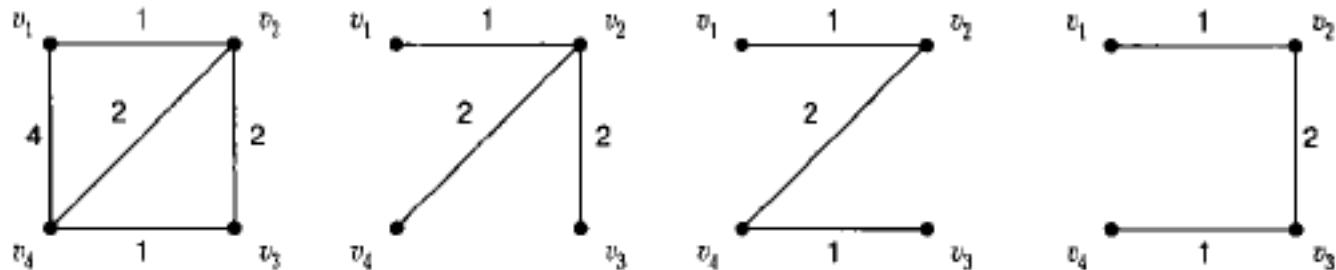
$$= \begin{vmatrix} 3 & -1 & 0 & -1 & 0 \\ -1 & 2 & -1 & 0 & 0 \\ 0 & -1 & 2 & -1 & 0 \\ -1 & 0 & -1 & 4 & 0 \\ 0 & 0 & 0 & -1 & 1 \end{vmatrix} + \begin{vmatrix} 3 & -1 & 0 & -1 & 0 \\ -1 & 2 & -1 & 0 & 0 \\ 0 & -1 & 2 & -1 & 0 \\ -1 & 0 & -1 & 4 & 0 \\ 0 & 0 & 0 & -1 & 2 \end{vmatrix} = - \begin{vmatrix} 3 & -1 & 0 & -1 \\ -1 & 2 & -1 & 0 \\ 0 & -1 & 2 & -1 \\ -1 & 0 & -1 & 4 \end{vmatrix} + \begin{vmatrix} 3 & -1 & 0 & 0 \\ -1 & 2 & -1 & 0 \\ 0 & -1 & 2 & 0 \\ -1 & 0 & -1 & -1 \end{vmatrix} +$$

$$+ 2 \cdot \begin{vmatrix} 3 & -1 & 0 & -1 \\ -1 & 2 & -1 & 0 \\ 0 & -1 & 2 & -1 \\ -1 & 0 & -1 & 4 \end{vmatrix} = 3 \cdot \begin{vmatrix} 3 & -1 & 0 \\ -1 & 2 & -1 \\ 0 & -1 & 2 \end{vmatrix} + \begin{vmatrix} -1 & 2 & -1 \\ 0 & -1 & 2 \\ -1 & 0 & -1 \end{vmatrix} + \begin{vmatrix} 3 & -1 & 0 \\ -1 & 2 & -1 \\ -1 & 0 & -1 \end{vmatrix} = 3(12 + 0 + 0 - 0 - 2 - 3) +$$

$$+ (-1 - 4 + 0 + 1 - 0 - 0) + (-6 - 1 + 0 - 0 + 1 - 1) = 11.$$



Примеры кратчайших остовов



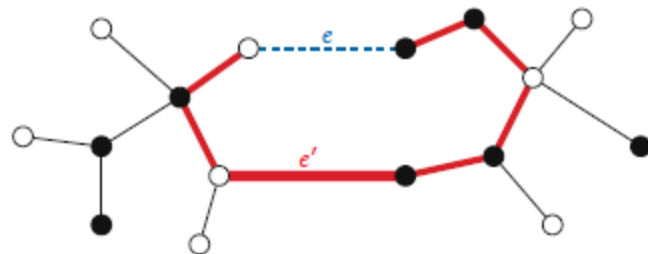
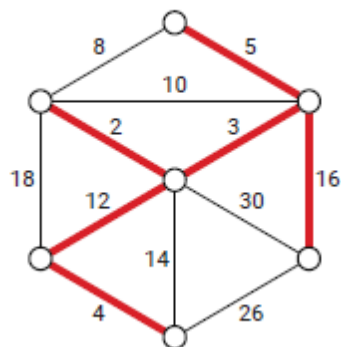
- Эта задача возникает при проектировании линий электропередачи, трубопроводов, дорог и т. п., когда требуется заданные центры соединить некоторой системой каналов связи так, чтобы любые два центра были связаны либо непосредственно соединяющим их каналом, либо через другие центры и каналы, и чтобы общая длина (или, например, стоимость) каналов связи была минимальной. В этой ситуации заданные центры можно считать вершинами полного графа с весами ребер, равными длинам (стоимости) соединяющих эти центры каналов.
- Тогда искомая сеть будет кратчайшим остовным подграфом полного графа. Очевидно, что этот кратчайший остовный подграф должен быть деревом. Поскольку полный граф K_n содержит n^{n-2} различных остовных деревьев, то решение этой задачи «слепым» перебором вариантов потребовало бы чрезвычайно больших вычислений даже при относительно малых n .

Алгоритмы построения кратчайшего остова

Для ее решения имеются эффективные алгоритмы, применимые к произвольному связному графу.

- **Алгоритм Дж. Краскала (1956 г.)** - перебираем рёбра в порядке возрастания их веса. Если ребро является безопасным, добавляем его. $O(E * \log(E))$.
- **Алгоритм Р. Прима (1957 г.)** - в каждый момент есть ровно одна нетривиальная компонента связности. На каждом шаге добавляем в неё безопасное ребро. $O(V^2)$ $O(E + V \log V)$
- **Алгоритм О. Борувки (Борувки-Соллина) (1926 г.?)** - добавляем все безопасные рёбра и производим рекурсивный вызов. $O(E \log V)$
- В некоторых ситуациях требуется построить остов не минимального, а максимального веса. К этой задаче также применимы и алгоритм Краскала, и алгоритм Прима. Следует только всюду минимальный вес заменить максимальным.
- ***Как найти остов графа с помощью поиска в ширину?***

Пример



Алгоритм Краскала

Вход: список E рёбер графа G с длинами, упорядоченный в порядке возрастания длин.

Выход: множество T рёбер кратчайшего остова.

$T := \emptyset$

$k := 1$ { номер рассматриваемого ребра }

for i **from** 1 **to** $p - 1$ **do**

while $z(T + E[k]) > 0$ **do**

$k := k + 1$ { пропустить это ребро }

end while

$T := T + E[k]$ { добавить это ребро в SST }

$k := k + 1$ { и исключить его из рассмотрения }

end for

- Исследование алгоритма Краскала дало толчок развитию теории жадных алгоритмов.
- Семейство всех таких подмножеств множества рёбер графа, которые не содержат циклов, является матроидом.
- Алгоритм Краскала как жадный алгоритм, применённый к матроиду, находит ациклическое подмножество рёбер наименьшего веса. По построению алгоритма (основной цикл до $p - 1$) это подмножество рёбер древочисленно, а значит, является кратчайшим остовом.

Пример

KRUSKAL(V, E):

sort E by increasing weight

$F \leftarrow (V, \emptyset)$

for each vertex $v \in V$

 MAKESET(v)

for $i \leftarrow 1$ to $|E|$

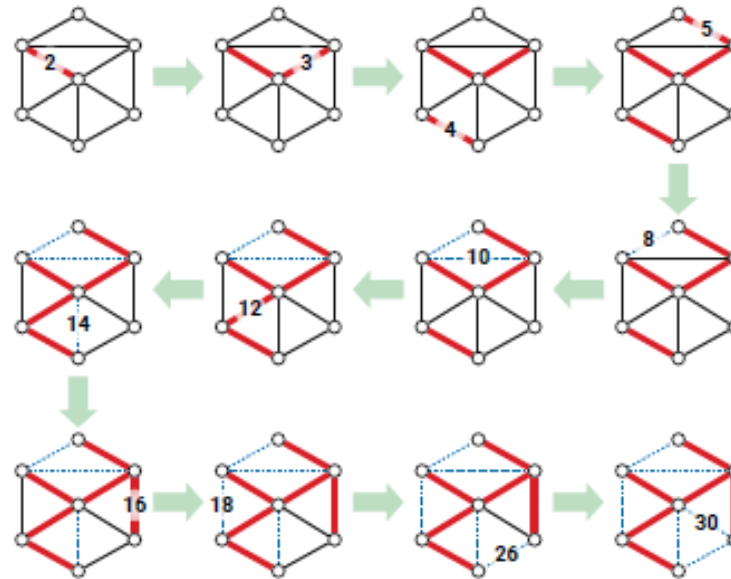
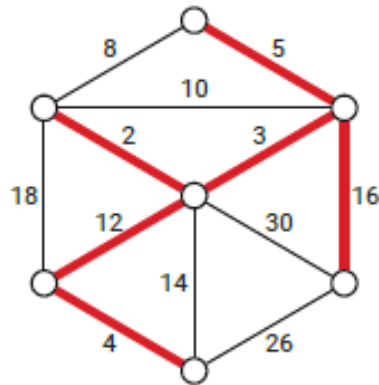
$uv \leftarrow$ i th lightest edge in E

 if FIND(u) $\neq$ FIND(v)

 UNION(u, v)

 add uv to F

return F



Алгоритм Прима (Ярника)

Вход: граф $G(V, E)$, заданный матрицей длин рёбер C .

Выход: кратчайший остов T .

$T := V$

while в T больше одного элемента **do**

 взять любое поддерево из T

 пайти к нему ближайшее

 соединить эти деревья в T

end while

Вход: граф $G(V, E)$, заданный матрицей длин рёбер C .

Выход: множество T рёбер кратчайшего остова.

select $u \in V$ { выбираем произвольную вершину }

$S := \{u\}$ { S — множество вершин, включённых в кратчайший остов }

$T := \emptyset$ { T — множество рёбер, включённых в кратчайший остов }

for $v \in V - u$ **do**

if $v \in \Gamma(u)$ **then**

$\alpha[v] := u$ { u — ближайшая вершина остова }

$\beta[v] := C[u, v]$ { $C[u, v]$ — длина соответствующего ребра }

else

$\alpha[v] := 0$ { ближайшая вершина остова неизвестна }

$\beta[v] := \infty$ { и расстояние также неизвестно }

end if

end for

for i **from** 1 **to** $p - 1$ **do**

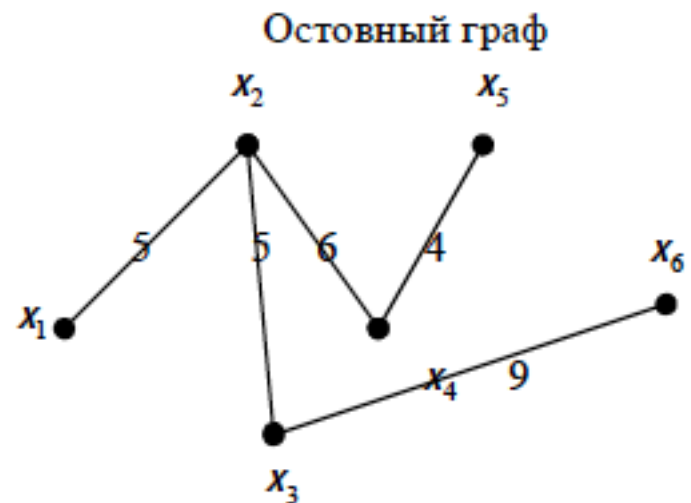
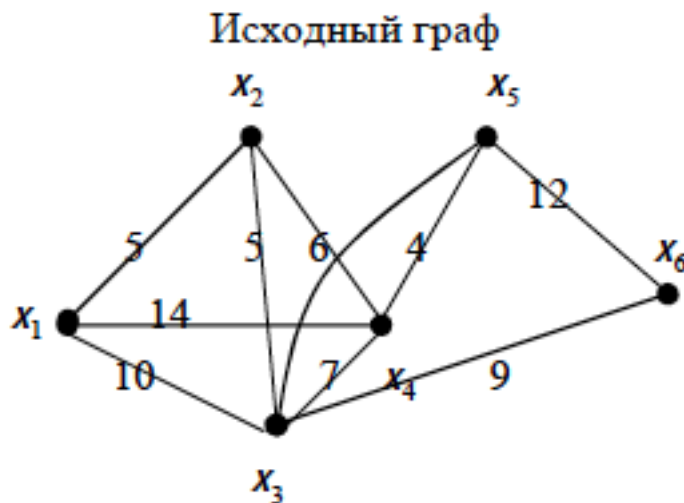
Алгоритм Прима

```
 $x := \infty$  { начальное значение для поиска ближайшей вершины }  
for  $v \in V \setminus S$  do  
  if  $\beta[v] < x$  then  
     $w := v$  { нашли более близкую вершину }  
     $x := \beta[v]$  { и расстояние до неё }  
  end if  
end for  
 $S := S + w$  { добавляем найденную вершину в остов }  
 $T := T + (\alpha[w], w)$  { добавляем найденное ребро в остов }  
for  $v \in \Gamma(w)$  do  
  if  $v \notin S$  then  
    if  $\beta[v] > C[v, w]$  then  
       $\alpha[v] := w$  { изменяем ближайшую вершину остова }  
       $\beta[v] := C[v, w]$  { и длину ведущего к ней ребра }  
    end if  
  end if  
end for  
end for
```

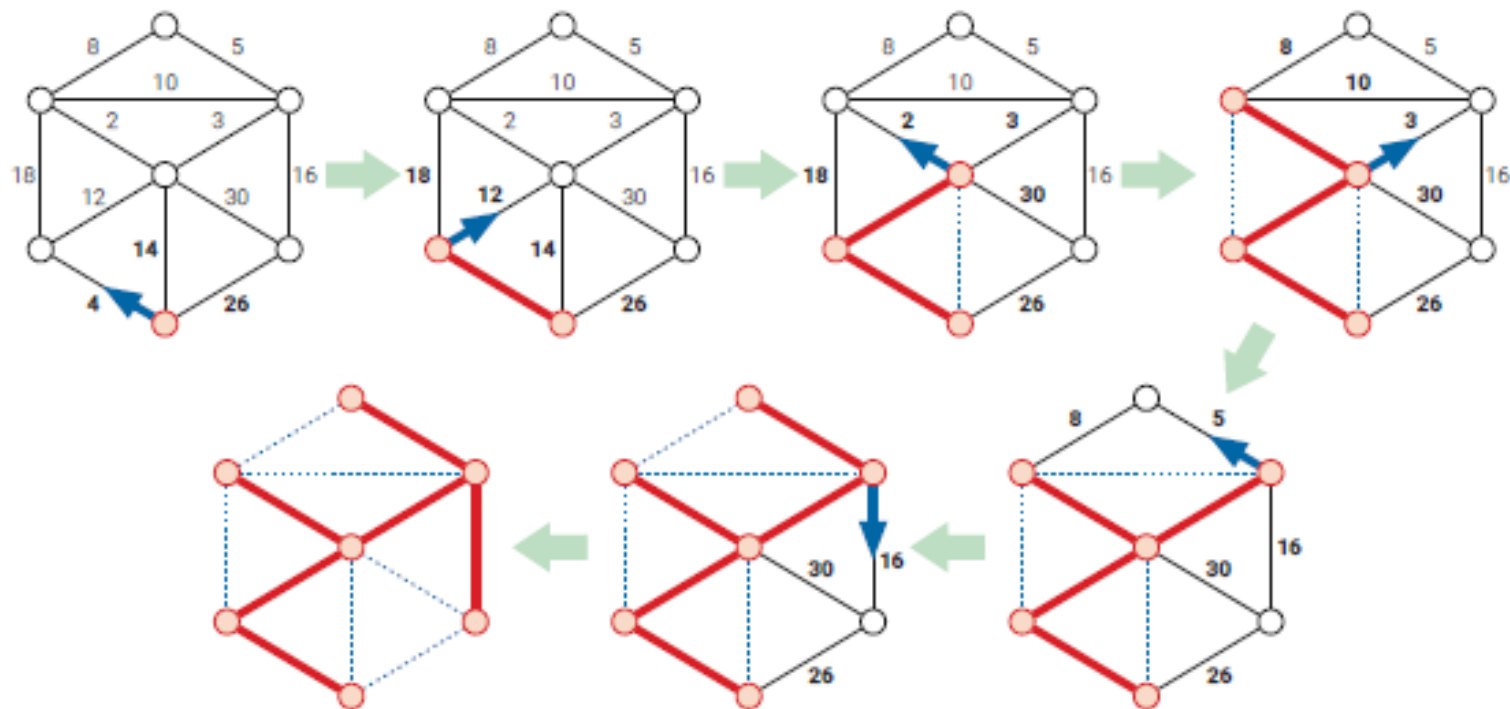
- Алгоритм Прима буквально следует схеме. В качестве первого из соединяемых деревьев используется одно и то же разрастающееся дерево T , а в качестве второго — ближайшая одиночная вершина.
- Алгоритм Прима отличается от алгоритма Краскала только тем, что на каждом этапе строится не просто ациклический граф, но дерево.

Пример

$$\Omega = \begin{matrix} & \begin{matrix} x_1 & x_2 & x_3 & x_4 & x_5 & x_6 \end{matrix} \\ \begin{matrix} x_1 \\ x_2 \\ x_3 \\ x_4 \\ x_5 \\ x_6 \end{matrix} & \begin{pmatrix} - & 5 & 10 & 14 & \infty & \infty \\ 5 & - & 5 & 6 & \infty & \infty \\ 10 & 5 & - & 7 & 8 & 9 \\ 14 & 6 & 7 & - & 4 & \infty \\ \infty & \infty & 8 & 4 & - & 12 \\ \infty & \infty & 9 & \infty & 12 & - \end{pmatrix} \end{matrix}$$



Пример



Алгоритм

JARNÍK(V, E, s):
JARNÍKINIT(V, E, s)
JARNÍKLOOP(V, E, s)

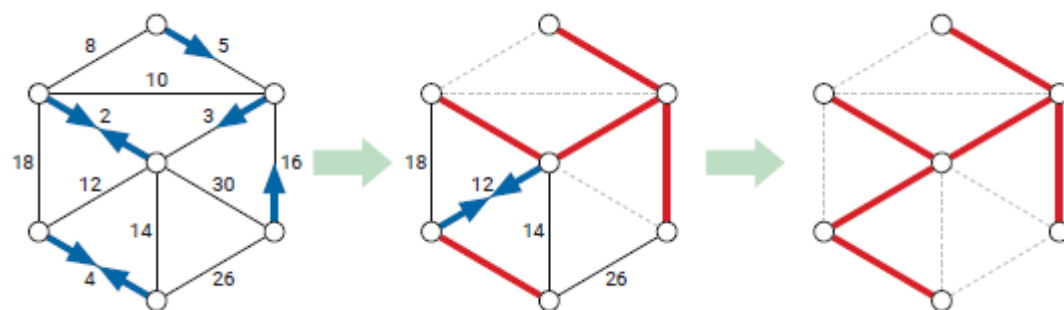
JARNÍKINIT(V, E, s):
for each vertex $v \in V \setminus \{s\}$
 if $vs \in E$
 $edge(v) \leftarrow vs$
 $priority(v) \leftarrow w(vs)$
 else
 $edge(v) \leftarrow \text{NULL}$
 $priority(v) \leftarrow \infty$
INSERT(v)

JARNÍKLOOP(V, E, s):
 $T \leftarrow (\{s\}, \emptyset)$
for $i \leftarrow 1$ to $|V| - 1$
 $v \leftarrow \text{EXTRACTMIN}$
 add v and $edge(v)$ to T
 for each neighbor u of v
 if $u \notin T$ and $priority(u) > w(uv)$
 $edge(u) \leftarrow uv$
 DECREASEKEY($u, w(uv)$)

Алгоритм Борувки

- Пусть T подграф графа G . Изначально T содержит все вершины из G и не содержит ребер.
- Будем добавлять в T ребра следующим образом:
- Пока T не является деревом для каждой компоненты связности находим минимальное по весу ребро, которое связывает вершину из данной компоненты с вершиной, не принадлежащей данной компоненте.
- Добавим в T все ребра, которые хотя бы для одной компоненты связности оказались минимальными.
- Получившийся граф T является минимальным остовным деревом графа G .
- Данный алгоритм может работать неправильно, если в графе есть ребра равные по весу. Например, полный граф из трех вершин, вес каждого ребра равен один. В T могут быть добавлены все три ребра. Избежать эту проблему можно, выбирая в первом пункте среди ребер, равных по весу, ребро с наименьшим номером.

Пример

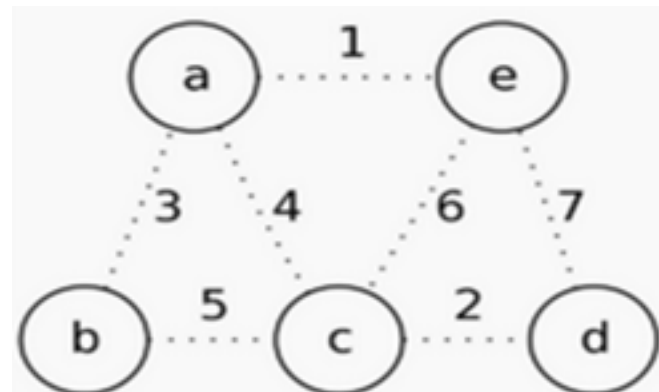


Алгоритм Борувки

```
BORUVKA( $V, E$ ):  
   $F = (V, \emptyset)$   
   $count \leftarrow \text{COUNTANDLABEL}(F)$   
  while  $count > 1$   
     $\text{ADDALLSAFEEDGES}(E, F, count)$   
     $count \leftarrow \text{COUNTANDLABEL}(F)$   
  return  $F$ 
```

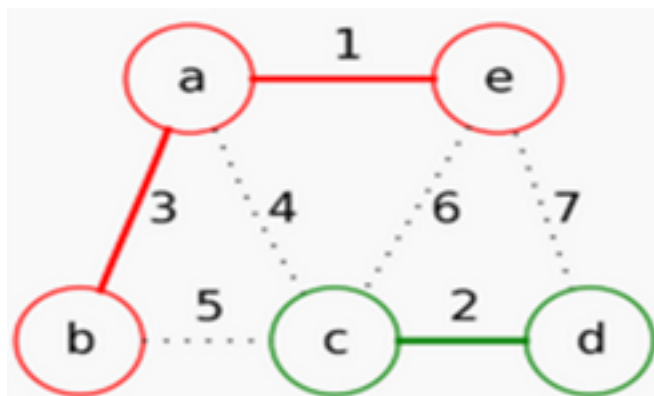
```
ADDALLSAFEEDGES( $E, F, count$ ):  
  for  $i \leftarrow 1$  to  $count$   
     $safe[i] \leftarrow \text{NULL}$   
  for each edge  $uv \in E$   
    if  $comp(u) \neq comp(v)$   
      if  $safe[comp(u)] = \text{NULL}$  or  $w(uv) < w(safe[comp(u)])$   
         $safe[comp(u)] \leftarrow uv$   
      if  $safe[comp(v)] = \text{NULL}$  or  $w(uv) < w(safe[comp(v)])$   
         $safe[comp(v)] \leftarrow uv$   
  for  $i \leftarrow 1$  to  $count$   
    add  $safe[i]$  to  $F$ 
```

Пример



Переберём все вершины и отметим для каждой вершины инцидентное ей ребро минимального веса.

Вершина	a	b	c	d	e
Ребро минимального веса	ae	ab	cd	cd	ae

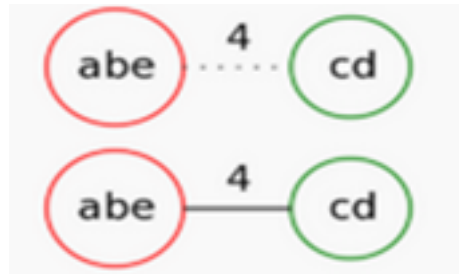


Объединим каждую полученную компоненту связности в одну вершину.

Полученные вершины *abe* и *cd* соединяют рёбра *bc*, *ac*, *ec* и *ed*.

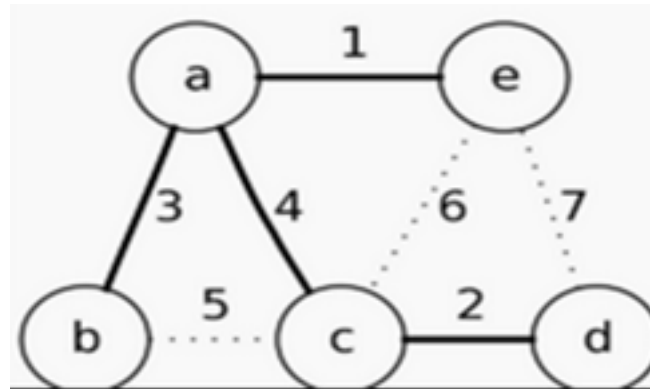
Выберем среди них ребро с минимальным весом - *ac* и положим его между полученными вершинами.

Алгоритм Борувки



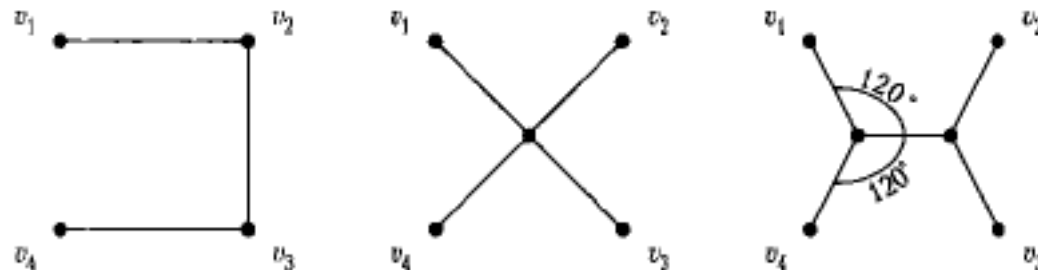
Граф сжат в одну вершину.

Теперь нужно заменить множество рёбер заданного графа на полученное в алгоритме.



Задача Штейнера

- Задача о нахождении кратчайшего остова принадлежит к числу немногих задач теории графов, которые можно считать полностью решёнными. Между тем, если изменить условия задачи (на первый взгляд даже незначительно), то она оказывается значительно более трудной.
- Пусть задано множество городов на плоскости и нужно определить минимальный (по сумме расстояний) набор железнодорожных линий, который позволил бы переехать из любого города в любой другой. Кратчайший остов полного графа расстояний между городами не будет являться решением этой задачи, известной как **задача Штейнера**.
- В задаче Штейнера допускается введение дополнительных вершин, называемых *точками Штейнера*.
- На рисунке приведены, соответственно, диаграммы кратчайшего остова, наивного «решения» задачи Штейнера и правильного решения для случая, когда города расположены в вершинах квадрата.



Евклидова задача Штейнера

- Произвольное множество U точек евклидовой плоскости требуется соединить непрерывными линиями так, чтобы любые две точки были связаны либо непосредственно соединяющей их линией, либо через другие точки и соединяющие их линии, и чтобы сумма длин линий была минимальной.
- Множеству точек U можно поставить в соответствие полный граф $K(U)$, вершинами которого будут элементы U , а вес каждого ребра будет равен расстоянию между соответствующими точками.
- Евклидова задача Штейнера отличается от задачи построения остова минимального веса в графе $K(U)$ тем, что в этот граф разрешается вносить новые вершины — точки Штейнера. Их можно добавлять столько, сколько потребуется, чтобы дерево, их соединяющее, имело минимальный вес.
- Если в качестве расстояния между точками (x_1, y_1) и (x_2, y_2) взять величину $|x_1 - x_2| + |y_1 - y_2|$, то получится прямоугольная задача Штейнера. Эта задача сводится к следующей широко известной задаче.
- Задача Штейнера на графах: В связном завершенном графе G с выделенным подмножеством вершин U в VG требуется найти связный подграф T , удовлетворяющий следующим двум условиям:
 - множество VT содержит заданное подмножество U ;
 - граф T имеет минимальный вес среди всех подграфов, удовлетворяющих условию
- Искомый подграф является деревом. Он называется **деревом Штейнера**.

Анализ задачи Штейнера

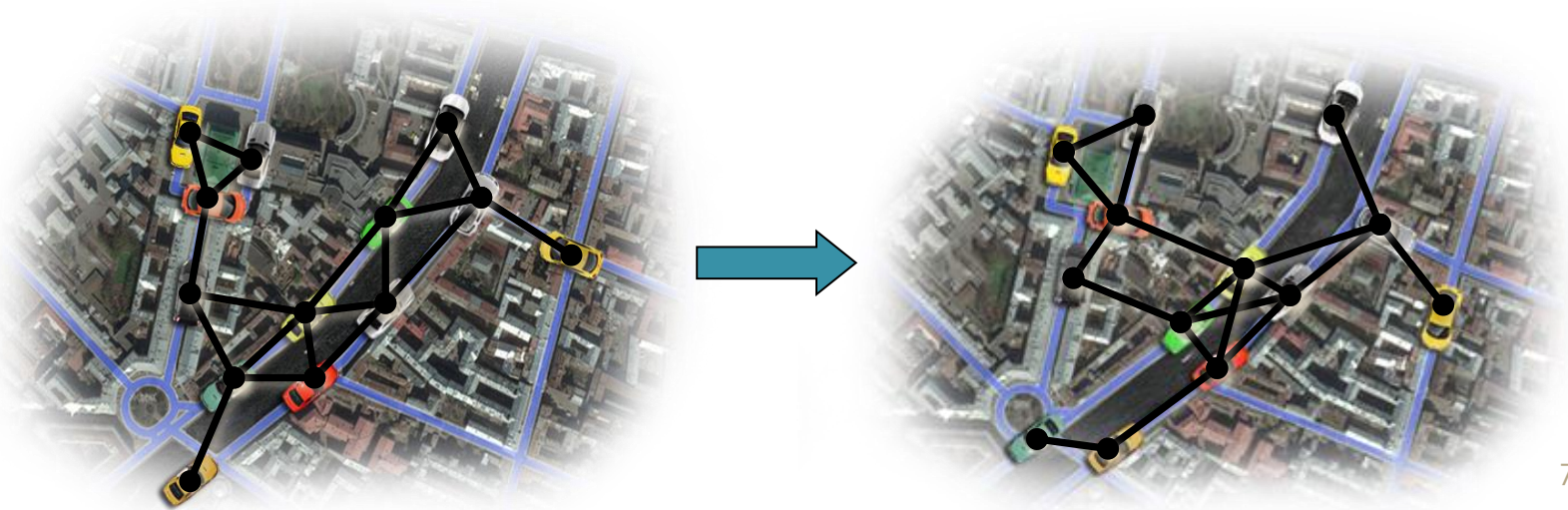
- Задача построения дерева Штейнера эквивалентна задаче *нахождения остова минимального веса в порожденных подграфах графа G , множества вершин которых содержат U .*
- Задача Штейнера на плоскости хорошо изучена, в частности, известно много важных свойств точного решения. Например, известно, что каждая точка Штейнера имеет степень 3 и отрезки в ней встречаются под углом 120° .
- Тем не менее до сих пор неизвестно достаточно эффективных алгоритмов решения данной задачи, и она остаётся предметом интенсивных исследований.

Пример: Алгоритм определения времени существования MESH-сети транспортных средств

Задача исследования: разработка алгоритма определения времени существования MESH-сети

Особенности MESH-сетей

- **Избыточность соединений**
- Децентрализованность организации
- Динамические изменения связей



Mesh-сеть

- **Mesh-сеть** — это распределенная, одноранговая, ячеистая сеть.

Каждый узел в ней обладает такими же полномочиями как и все остальные — все узлы в сети равны.

Сети бывают самоорганизующиеся и настраиваемые, первый тип сетей при включении оборудования, которое его поддерживает, автоматически подключаются к существующим участникам, выбирают оптимальные маршруты и самонастраиваются внутри сети.

Полноценная Mesh Wi-Fi сеть это такая сеть, для подключения к которой не требуется никакого дополнительного ПО *кроме dhcp-клиента и поддержки ipv6 системой.*

- Нет единого центра для получения IP адресов (DHCP), Маршруты в сети полностью распределенные и динамические. Объединение сетей происходит в автоматическом режиме...

Анализ алгоритма

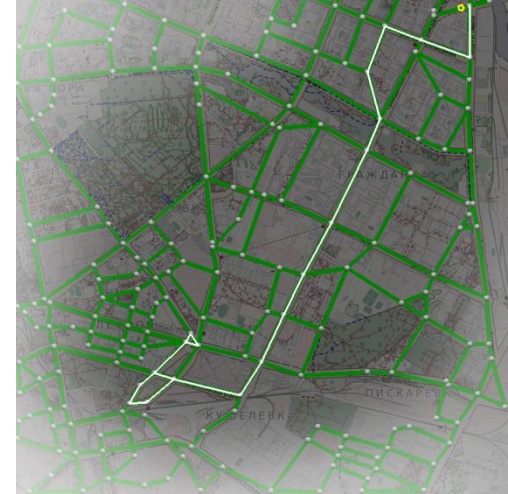
На основе данных о времени существования MESH-сети:

Определение возможности передачи данных в ЦОД от любого узла сети

- Ограничения:
 - Перемещение ТС в пределах дорожной сети
 - Заданные маршруты движения ТС и скорости перемещения на каждом участке
- Анализ непрерывного процесса перемещения ТС
- Определение моментов времени разрывов соединений между ТС
- Поиск вариантов для восстановления связности

Сужение области поиска:

- использование остовного дерева графа связей



Оценка сложности работы алгоритма

- Построение графа расстояний между ТС – $O(N^2)$
- Построение остовного дерева связей – $O(E \cdot \text{Log}(E))$
- Анализ разрывов соединений
 - Поиск разрывов соединений - $O(k)$
 - Поиск вариантов восстановления связности – $O(N^2)$

Общая сложность работы алгоритма: $O(kN^2)$



СПАСИБО
ЗА ВНИМАНИЕ